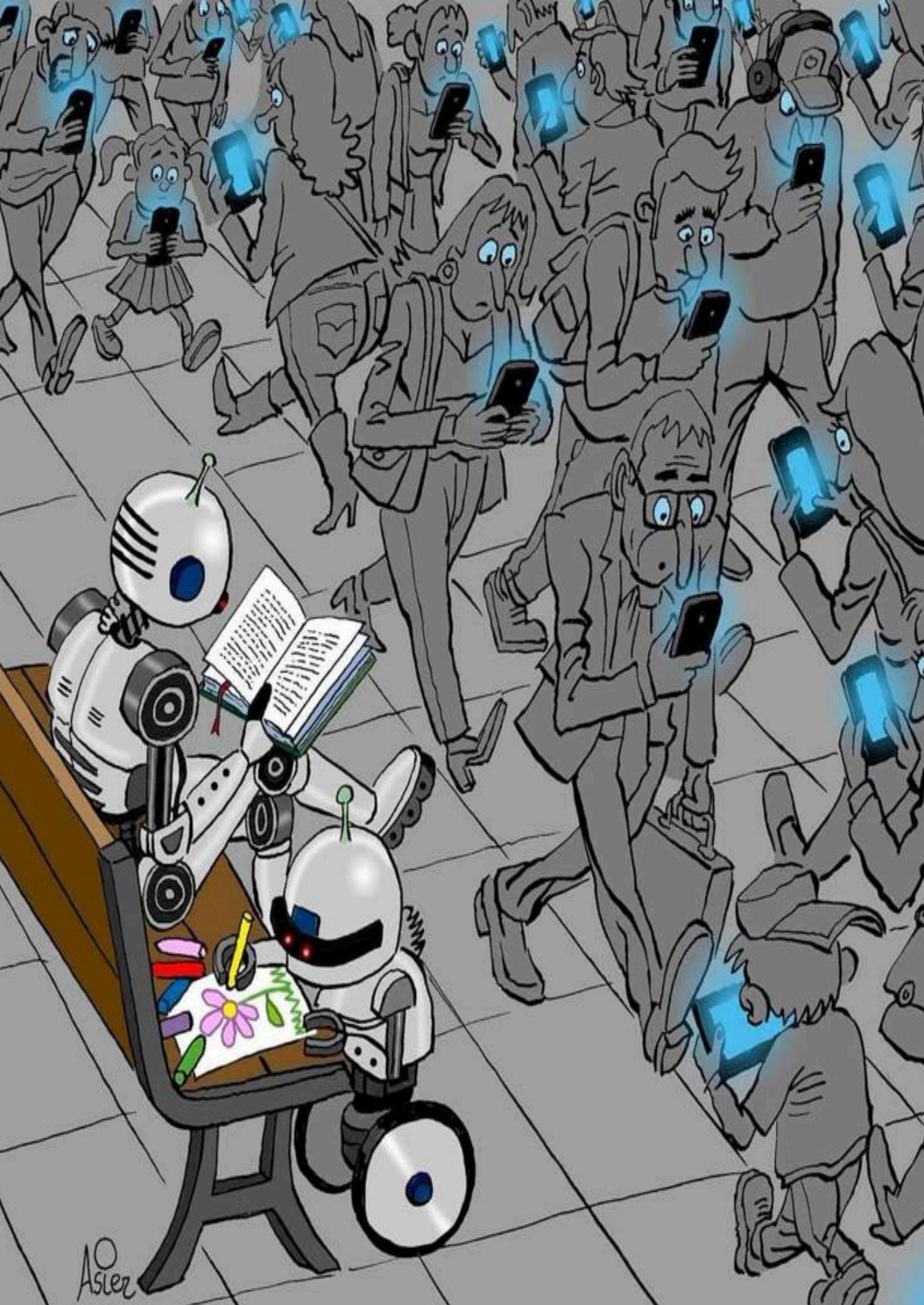


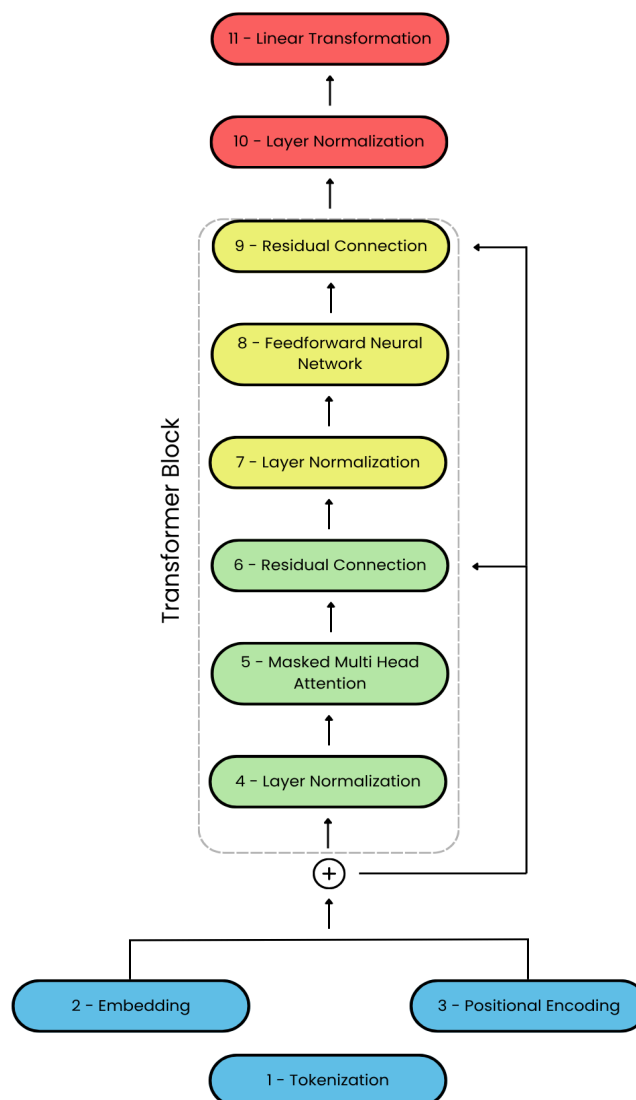


Go Forward and Transform

Arquitetura Transformer e a Evolução dos Modelos de Linguagem Modernos

Eliezer Carvalho

eliezer.carvalho.11.8a@gmail.com



Abstracto

Os Transformers [1] constituem uma arquitetura de Redes Neurais Artificiais fundamentada em mecanismos de Attention. Num período recente, estes modelos estabeleceram-se como a base dominante no Processamento de Linguagem Natural, impulsionando o desenvolvimento de *Large Language Models* (LLMs) com capacidades avançadas de geração e compreensão textual.

O presente estudo propõe uma exploração e compreensão da arquitetura dos Transformers. É apresentada uma visão abrangente do ecossistema contemporâneo de LLMs, incluindo técnicas e ferramentas fundamentais neste domínio. Este projeto apresenta-se como um guia teórico para a introdução ao Processamento de Linguagem Natural (NLP), permitindo a aquisição de uma compreensão sólida das tecnologias subjacentes aos modelos de linguagem modernos. Por fim, são abordadas tendências atuais e futuras na área, bem como as suas implicações no desenvolvimento de sistemas inteligentes.

1. Processamento de Linguagem Natural	4
1.1 Sistemas Baseados em Regras (1950 - 1970)	4
1.2 Revolução Estatística (1980 - 1990)	5
1.3 Era do Deep Learning (2000 - Presente)	6
2. Transformers	7
2.1 Pre-Training	8
2.2 Arquitetura dos Transformers	10
2.2.1 Tokenization	10
2.2.2 Sequence Length e Batch Size	10
2.2.3 Embeddings	11
2.2.4 Positional Encoding	12
2.2.5 Attention	14
2.2.5.1 Scaled Dot-Product Attention	14
Self Attention	15
Masked Self Attention	15
Cross Attention	15
2.2.5.2 Multi-Head Attention	16
2.2.6 Add & Norm	17
Residual Connections (Add)	17
Layer Normalization (Norm)	18
Post-Layer Norm Transformer Design	18
Pre-Layer Norm Transformer Design	19
2.2.7 Feedforward Neural Network	20
2.2.8 Language Modelling / Output Projection & Softmax	20
2.2.9 Treino	21
2.2.9.1 Gradientes e Backpropagation	21
2.2.9.2 Loss Function	22
2.2.9.3 Otimizadores	23
2.2.10 Inferência	24
Greedy Decoding	24
Beam Search	24
Top-k Sampling	24
Top-p Sampling	24
Temperature Sampling	24
2.3 Fine-Tuning	25
Full Fine-Tuning	26
Parameter-Efficient Fine-Tuning (PEFT)	26
Instruction Fine-Tuning	26
Reinforcement Learning from Human Feedback (RLHF)	26
2.4 Fine-Tuning vs Pre-Training	27
3. Large Language Models	28
3.1 Scaling Laws	29
3.2 Capacidades Emergentes em Large Language Models	30
3.3 Limitações dos Large Language Models	31

3.3.1 Alucinações (Hallucinations)	31
3.3.2 Limitação de Contexto (Context Window)	31
3.3.3 Data Dependence e Custos Computacionais	32
3.4 Large Language Models como Foundation Models	33
3.5 Avaliação de Large Language Models - Benchmarks	34
4. Técnicas de Fine-Tuning	35
4.1 Fine-Tuning para Tarefas Específicas	35
4.2 Fine-Tuning para Domínios Específicos	36
4.3 Técnicas de Parameter-Efficient Fine-Tuning (PEFT)	37
4.3.1 Adapters	38
4.3.2 LoRA (Low-Rank Adoption)	39
5. Prompt Engineering	41
5.1 Zero-Shot Prompting e Few-Shot Prompting	41
5.2 Chain-of-Thought (CoT)	42
Zero-Shot CoT	42
Few-Shot CoT	42
5.3 Role Prompting	43
6. Retrieval Augmented Generation (RAG)	44
6.1 How To RAG	45
6.2 RAG vs Fine-Tuning	47
7. Model Compression	48
7.1 Quantization	48
7.2 Pruning	49
Unstructured Pruning	49
Structured Pruning	49
7.3 Knowledge Distillation	50
7.4 Trade-Off entre Velocidade e Qualidade	51
8. Small Language Models	51
9. Sistemas e Arquiteturas	52
9.1 Arquitetura Geral de um Produto LLM	52
Input do Utilizador	52
Prompt Processing	52
RAG (Opcional)	52
Construção de Contexto	52
Inferência	52
Tool Calling	53
Memória	53
Geração de Resposta	53
9.2 AI Agents	53
9.3 Infraestrutura da Inferência	54
9.3.1 GPUs e Custos Computacionais	54
9.3.2 Técnicas para Mitigar Custos Computacionais na Inferência	55
Batching	55
KV Cache	55

9.3.3 Importância da Inferência	56
10. Deployment e Produção - Servidores, Web e Cloud Computing	56
10.1 Servidores	56
10.2 Web Deployment (API's)	57
10.3 Cloud Computing	57
11. Ferramentas e Tecnologias Importantes	58
Linguagens de Programação e Frameworks	58
Ambientes de Desenvolvimento	58
Ecossistema de Modelos	58
Fine-Tuning	58
Prompt Engineering	58
RAG	58
Model Compression	58
Inferência	58
Hospedagem	58
AI Agents	58
12. Tecnologias Emergentes	59
Small Language Models	59
Agentic AI	59
Multimodal Models	59
Mixture of Experts (MoE)	59
Long Context Models	59
RAG Evoluído	59
Model Compression + Edge IA	59
Modelos Self-Improving	59
Geração de Dados Sintéticos	59
Model Routing & Multi-Model Systems	59
AI Systems + API's	59

1. Processamento de Linguagem Natural

O Processamento de Linguagem Natural (PLN) é uma área da Ciência da Computação que estuda o desenvolvimento de sistemas capazes de analisar, reconhecer, interpretar e/ou gerar textos em linguagens humanas [2]. Esta área possibilita que computadores compreendam e interajam com a linguagem natural utilizada na comunicação quotidiana.

Em oposição às linguagens de programação, a linguagem natural [3] é inerentemente ambígua e dependente de contexto. Fatores como variações linguísticas, evolução da linguagem ao longo do tempo e diferenças regionais conferem ao Processamento de Linguagem Natural um caráter complexo e multifacetado. A ambiguidade lexical, a estrutura gramatical e a interpretação semântica são problemas que exigem modelos altamente sofisticados.

1.1 Sistemas Baseados em Regras (1950 - 1970)

As primeiras abordagens ao Processamento de Linguagem Natural eram predominantemente baseadas em *Sistemas Rule-Based*, recorrendo a conjuntos de regras previamente definidas por especialistas [4]. Estes sistemas possibilitam unicamente a resposta a entradas altamente específicas, dependendo, em larga escala, de comandos ou padrões previamente programados. Tais abordagens apresentam limitações significativas, designadamente a incapacidade de aprendizagem a partir de dados e a reduzida escalabilidade. Como resultado, os sistemas manifestavam-se rígidos, difíceis de generalizar e incapazes de lidar com a complexidade e variabilidade presentes na linguagem natural. Estes sistemas são muito abordados em contexto universitário como introdução à Inteligência Artificial.



Figura 1 - Sistema Baseado em Regras

1.2 Revolução Estatística (1980 - 1990)

O PLN estatístico emergiu como uma evolução das abordagens baseadas em regras, incorporando métodos de Machine Learning para a análise de texto e fala [5]. Nesta abordagem, os sistemas passam a extrair, classificar e rotular automaticamente elementos linguísticos, atribuindo-lhes probabilidades com base em modelos estatísticos. A Revolução Estatística introduziu igualmente a noção de representar elementos da linguagem, em termos quantitativos, tornando possível a sua modelação por meio de métodos estatísticos, como a regressão ou modelos de Markov [6].

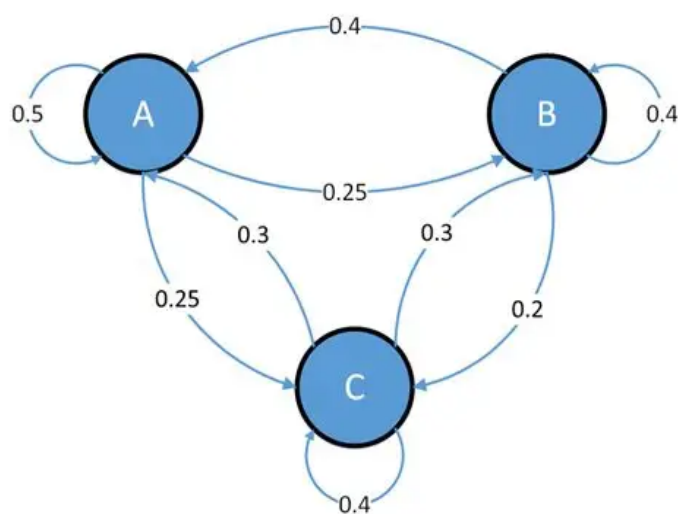


Figura 2 - Exemplo simples da Cadeia de Markov

1.3 Era do Deep Learning (2000 - Presente)

Modelos baseados em Deep Learning tornaram-se a abordagem dominante no Processamento de Linguagem Natural, recorrendo a grandes volumes de **dados não estruturados** [7] para melhorarem progressivamente o seu desempenho. O Deep Learning representa uma evolução dos métodos estatísticos tradicionais, distinguindo-se pela utilização de Redes Neurais Profundas [8] com a capacidade de relacionar padrões não lineares [9].

A adoção das Redes Neurais revolucionou o PLN, resultando em arquiteturas como Redes Neurais Recorrentes (RNNs), Long Short-Term Memory (LSTM) e, mais recentemente, Transformers, que constituem a base de modelos como BERT e GPT.

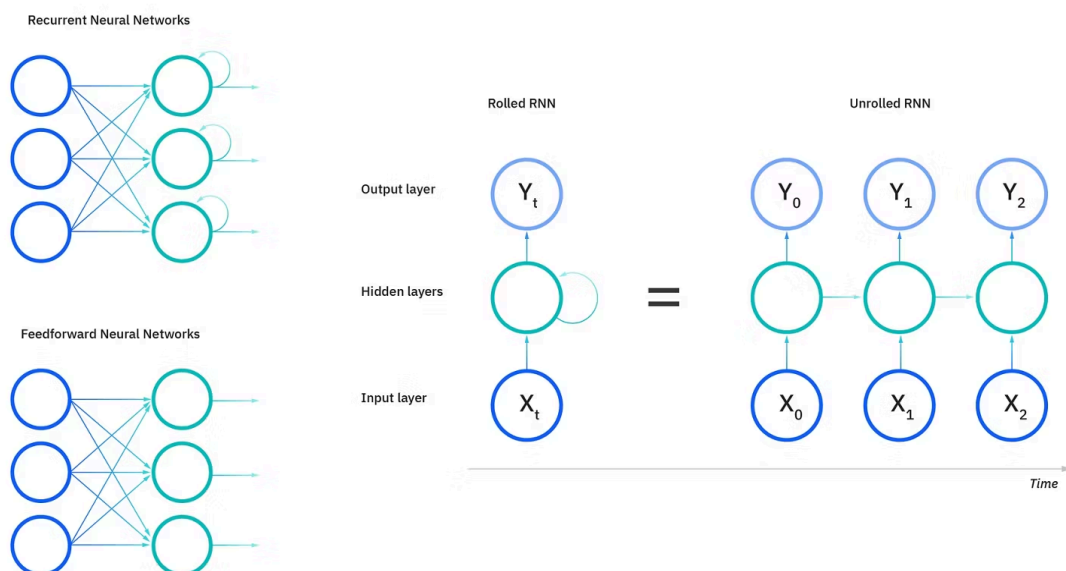


Figura 3 - Feedforward Neural Networks vs Recurrent Neural Networks

2. Transformers

As Recurrent Neural Networks (RNNs) [10] foram uma das primeiras abordagens eficazes para o processamento de dados sequenciais no domínio do PLN. Estas Redes Neurais apresentam uma característica particular para a modelação de dependências temporais, uma vez que processam a informação de forma sequencial, mantendo um estado interno que incorpora o contexto anterior. As RNNs apresentam limitações significativas durante o processo de treino. Entre os principais desafios, destacam-se os problemas de *vanishing gradient* [11] e *exploding gradient* [12]. Variantes como as Long Short-Term Memory (LSTM) foram propostas para mitigar estes problemas, mas as RNN continuam a enfrentar dificuldades na modelação eficiente de contextos longos e na paralelização do treino.

Os Transformers consistem numa arquitetura de Redes Neurais que mudou o paradigma da Inteligência Artificial por completo. A introdução destes modelos ocorreu em 2017, no artigo "[Attention Is All You Need](#)", e desde então, transformaram-se na abordagem “rainha” em modelos de Deep Learning. Nos tempos que correm, os Transformers constituem a pedra angular de modelos generativos de texto, como o **GPT**, **LLaMA** e o **Gemini**.

No âmbito da geração de texto, os modelos baseados em Transformers operam geralmente segundo o princípio de **next-token prediction** [13], ou seja, dado um determinado prompt, o modelo estima a probabilidade do próximo token (palavra ou subpalavra) mais adequado à sequência. A inovação mais impactante desta arquitetura vive no mecanismo de *Attention* [14], que torna possível o processamento de sequências completas em paralelo e a captura de dependências de longo alcance de forma mais eficiente em comparação com RNN's.

A família dos modelos GPT-2 [15] representa um exemplo desta abordagem, na medida em que ilustra os fundamentos em que se baseiam os Transformers. Apesar de não representarem o estado da arte contemporânea, os modelos desta família preservam os componentes fundamentais das arquiteturas modernas, estabelecendo uma base sólida para a compreensão dos conceitos subjacentes.

2.1 Pre-Training

Pre-Training corresponde à fase de treino dos modelos baseados em Transformers. O modelo adquire competências em representações gerais da linguagem a partir de grandes volumes [16] de dados não estruturados. Esta etapa é realizada de forma auto-supervisionada, ou seja, sem necessidade de intervenção humana. Consoante a arquitetura, são utilizados diferentes objetivos de treino.

Em **modelos autorregressivos ou decoders**, como os da família GPT, o modelo é treinado através da tarefa de **next-token prediction**, isto é, aprende a prever o próximo elemento de uma sequência com base no contexto anterior.

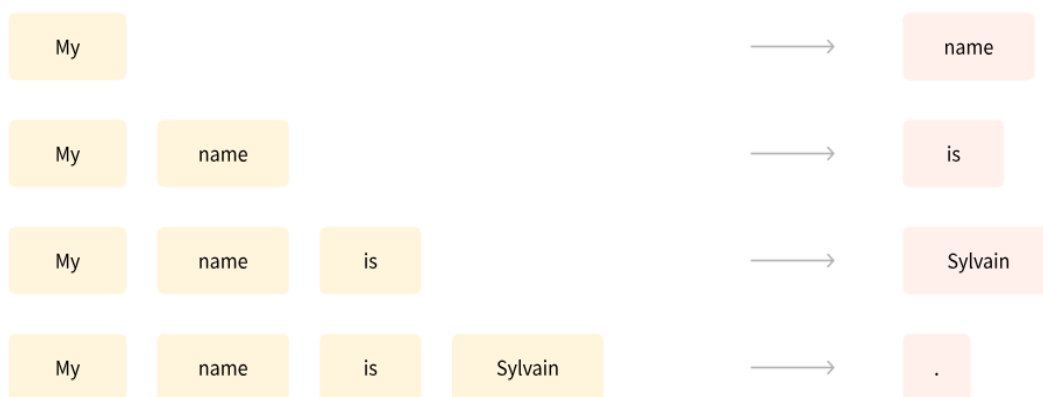


Figura 4 - Transformer Autoregressivo ou Decoder

Modelos bidirecionais ou encoders, como o BERT [17], empregam estratégias como Masked Language Modeling (MLM), em que partes da sequência de entrada são ocultadas e o modelo aprende a reconstruí-las com base no contexto circundante. Cenário perfeito para a tradução.

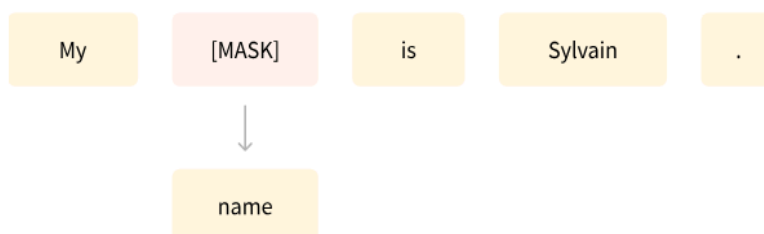


Figura 5 - Transformer Bidirecional ou Encoder

O propósito do Pré-Treino é facultar ao modelo a aquisição de competências linguísticas gerais, abrangendo sintaxe, semântica e relações contextuais, que subsequentemente poderão ser ajustadas a tarefas particulares por meio do fine-tuning (abordado mais à frente).

<i>Tarefa</i>	<i>Arquitetura Sugerida</i>	<i>Exemplos</i>
Classificação de Texto (sentimentos, tópicos)	Encoder	BERT, RoBERTa
Geração de Texto	Decoder	GPT, LLaMA
Tradução	Encoder-Decoder	T5, BART
Sumarizar	Encoder-Decoder	BART, T5
Responder a perguntas (extrair)	Encoder	BERT, RoBERTa
Responder a perguntas (gerar)	Encoder-Decoder ou Decoder	T5, GPT
Conversar	Decoder	GPT, LLaMA

Tabela 1 - Como escolher a melhor arquitetura ?

2.2 Arquitetura dos Transformers

Um Transformer é constituído por:

2.2.1 Tokenization

Tokenization [18] consiste no processo de decomposição do dataset em unidades menores, denominadas **tokens**. Os tokens podem corresponder a **palavras completas** (*full-sentence tokenizer*), **subpalavras** (*sub-words tokenizer*) ou até **caracteres individuais** (*char-level tokenizer*), dependendo do esquema de tokenização adotado.

Por exemplo, palavras como “dados” e “visualizar” podem ser representadas como tokens únicos, enquanto outras, como “empreendedor”, podem ser divididas em múltiplos tokens, de forma a melhor capturar padrões linguísticos e reduzir o tamanho do vocabulário. Muita atenção a esta fase, pois datasets com linguagem complexa necessitam Tokenizers com grande capacidade de Tokenization.

O conjunto total de tokens únicos denominado por **vocabulário** (*vocab_size*) é definido previamente ao treino do algoritmo de Tokenização. Por exemplo, o modelo GPT-2 utiliza um vocabulário com **50.257** tokens distintos.

Após a Tokenização, cada token é associado a um identificador único (*token ID*), que é posteriormente convertido num tensor [19].

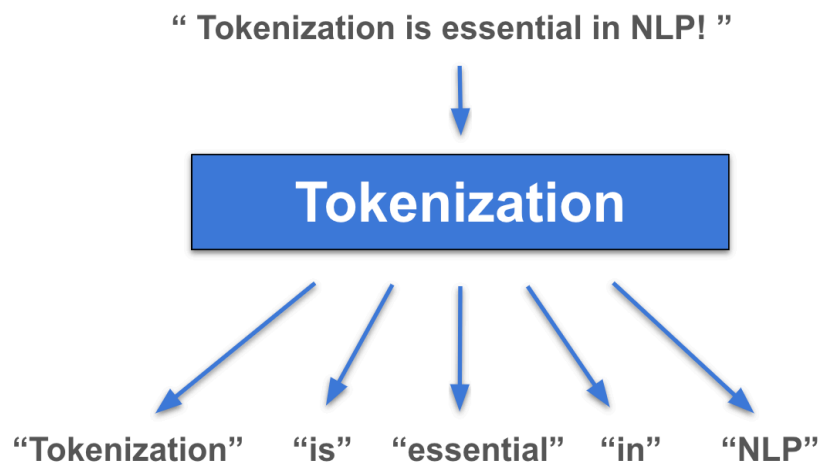


Figura 6 - Tokenization

2.2.2 Sequence Length e Batch Size

Transformers não são capazes de processar um dataset completo de forma simultânea. Esta limitação está associada ao custo computacional. Sendo assim, os dados são organizados em

excertos (ou **lotes**, em Inglês **batch**), possibilitando o processamento simultâneo de múltiplas sequências.

A entrada do modelo é tipicamente representada por um tensor de dimensão (**Batch Size**, **Sequence Length**). Muito Importante!

- **Batch Size** → define o número de sequências processadas em paralelo, influenciando diretamente a eficiência computacional e o consumo de memória.
- **Sequence Length** → determina o contexto máximo que o modelo é capaz de considerar em cada passo.

2.2.3 Embeddings

Os Embeddings são representações vetoriais de dados, utilizadas para converter informação simbólica, como tokens, em formato numérico suscetível de processamento. Os tokens são mapeados para vetores num espaço contínuo de dimensão fixa (n_embed). Estas representações permitem a captura de propriedades semânticas e sintáticas da linguagem, possibilitando a proximidade entre tokens com significados semelhantes nesse espaço.

Os embeddings possibilitam uma generalização mais eficiente, uma vez que codificam as relações entre palavras de forma implícita. Esta característica é fundamental para o desempenho de modelos baseados em Transformers [20].

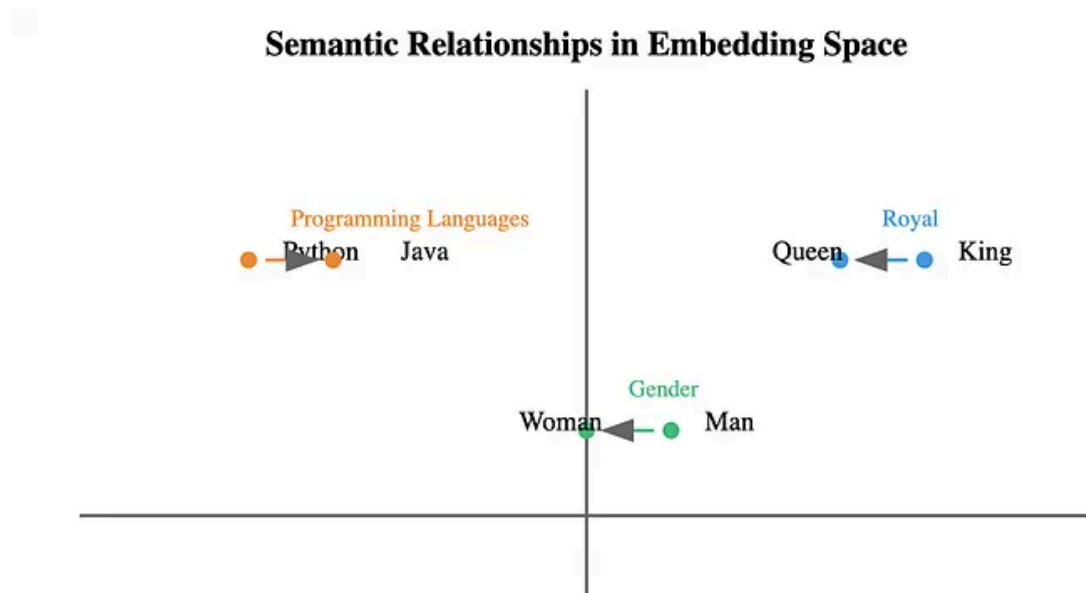


Figura 7 - Relações Semânticas no Espaço de Embedding

Tokenization vs Embeddings

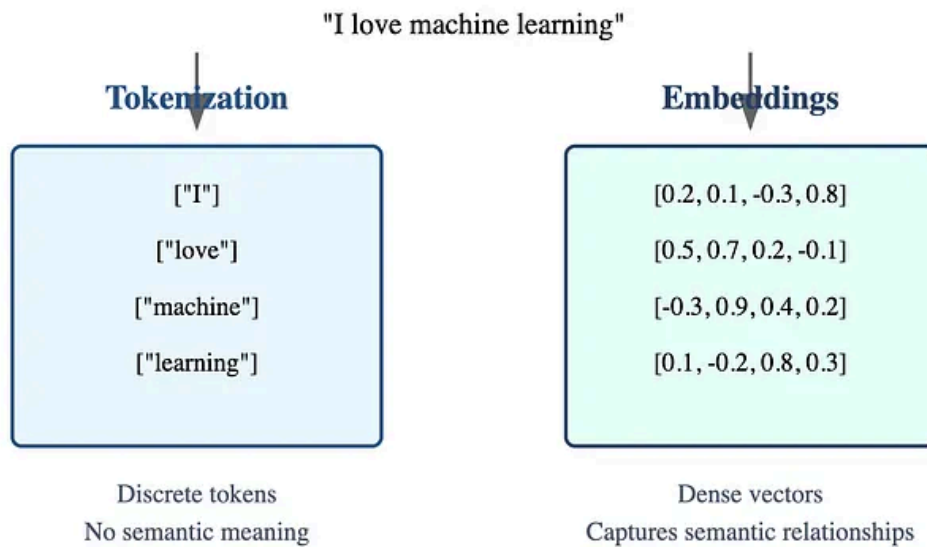


Figura 8 - Tokens → Embeddings

2.2.4 Positional Encoding

A mesma sequência de palavras, quando posicionada de forma diversa, podem assumir significados completamente distintos? Claro que sim!

Por exemplo, as frases "*o cão persegue o porco*" e "*o porco persegue o cão*", apresentam significados completamente distintos. Sem Positional Encoding, são tratadas de forma igual, uma vez que são percebidas como um conjunto desordenado de tokens. Com o propósito de preservar a informação sequencial e, por conseguinte, o seu significado, são integrados no modelo **Positional Encodings**.

De acordo com a proposta original, foi adotado um **Positional Encoding absoluto** fundamentado em funções sinusoidais, não sujeito a treino, facultando ao modelo a inferência de relações posicionais entre elementos distintos da sequência.

No entanto, as arquiteturas modernas de Transformers generativos passaram a empregar **Positional Encoding treináveis**, capazes de adaptar a representação posicional ao processo de treino.

Em tempos recentes, foram introduzidas abordagens como o *Rotary Positional Embedding (RoPE)* [21], que contribuíram para uma melhor modelação de dependências de longo alcance e para uma maior utilização generalizada em Large Language Models contemporâneos.

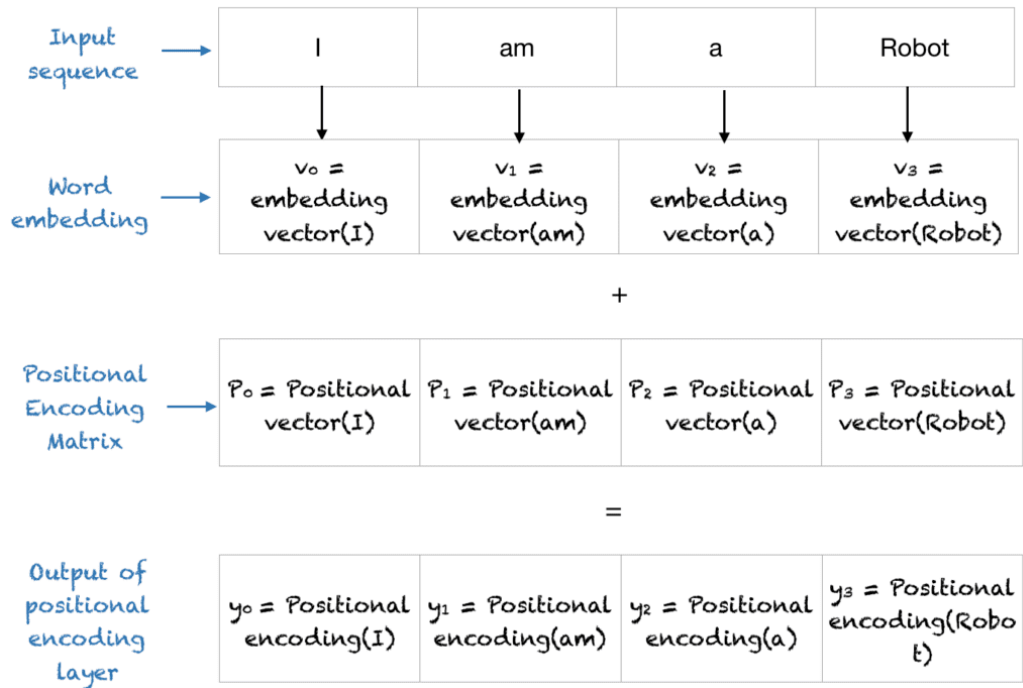


Figura 9 - Embedding e Positional Encoding

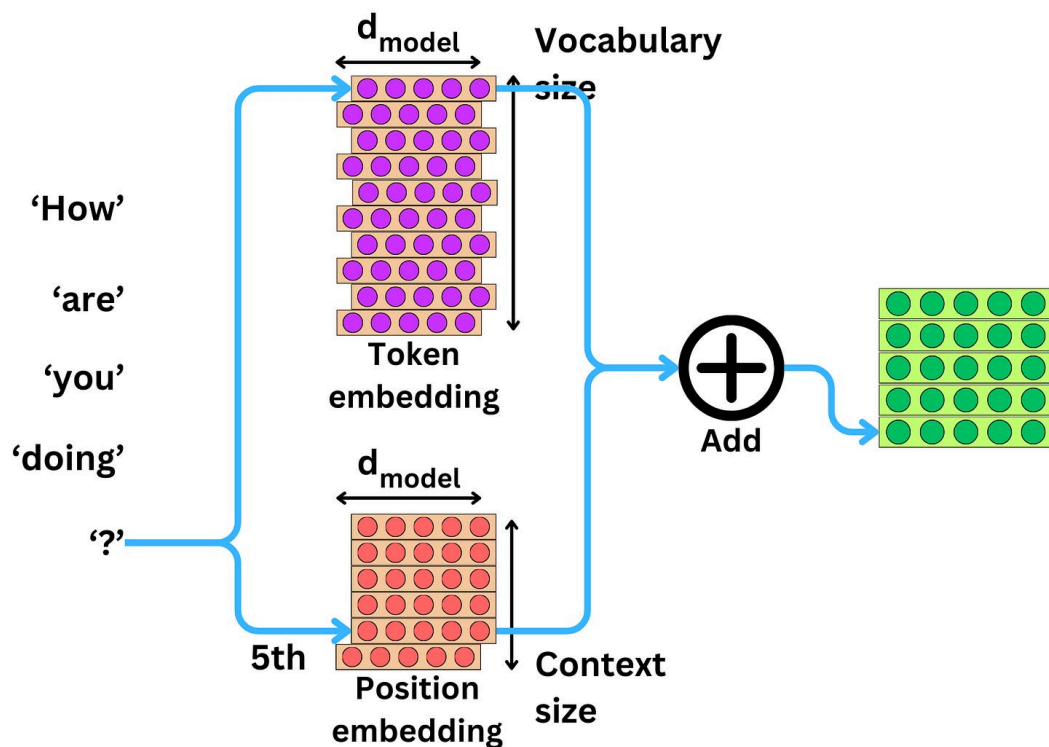


Figura 10 - Embedding + Positional Encoding

2.2.5 Attention

Attention constitui a componente central da arquitetura Transformer, sendo responsável pela capacidade do modelo em identificar relações importantes entre os diferentes tokens de uma sequência. De forma análoga à atenção humana, ao ler uma frase não se analisa cada palavra isoladamente, mas sim em função do seu contexto de forma a compreender o significado. O mecanismo de Attention permite ao modelo considerar simultaneamente os restantes elementos da sequência, atribuindo diferentes níveis de relevância a cada um deles.

O modelo é capaz de capturar dependências contextuais, incluindo relações de longo alcance, que se revelaram difíceis de modelar em RNN's e seus derivados. Existem diferentes variantes do mecanismo de Attention, dependendo da forma como as relações entre sequências são estabelecidas.

Vamos por partes..

2.2.5.1 Scaled Dot-Product Attention

Scaled Dot-Product Attention constitui o mecanismo utilizado pelos Transformers para o cálculo da relevância entre os diferentes tokens de uma sequência. Este processo fundamenta-se em três representações vetoriais fundamentais:

<i>Query (Q)</i>	<i>Key (K)</i>	<i>Value (V)</i>
O que ando à procura	O que represento	Que informação trago comigo

Tabela 2 - Q, K e V

Para cada token, o modelo efetua a comparação do vetor Query com todos os vetores Key da sequência através do produto escalar (*dot product*), obtendo assim, uma medida de similaridade entre os elementos. Estes valores correspondem aos **attention scores**, que indicam o grau de importância que cada token deve ter relativamente ao token atualmente processado. Os valores de classificação são normalizados através da função **softmax** [22], produzindo uma distribuição de probabilidades, que de seguida é utilizada para calcular uma combinação ponderada dos vetores Value.

A fórmula é elegantemente simples:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaled Dot-Product Attention

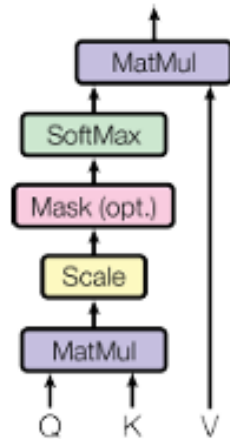


Figura 11 - Scaled Dot-Product Attention

<i>Self Attention</i>	<i>Masked Self Attention</i>	<i>Cross Attention</i>
Scaled Dot-Product Attention.	Variante da Self-Attention, que utiliza a técnica de <i>casual masking</i>. Utilizada predominantemente em modelos generativos auto regressivos. O objetivo principal consiste em impedir que o modelo acesse tokens futuros durante o processo de treino e inferência.	Mecanismo implementado em arquiteturas do tipo Encoder-Decoder, permitindo que uma sequência processe informação proveniente de outra sequência distinta. Extensamente aplicada em tarefas de tradução automática, sumarização e sistemas multimodais.

Tabela 3 - Tipos de Attention

2.2.5.2 Multi-Head Attention

A Multi-Head Attention representa uma extensão do mecanismo de Self-Attention, possibilitando ao modelo a capacidade de processar múltiplas representações de Attention de forma paralela [23]. Em vez de calcular uma única matriz de Attention, o modelo divide as representações de Query, Keys e Value em vários subconjuntos independentes, designados por "**attention heads**". Cada head aprende diferentes padrões e relações contextuais entre os tokens da sequência. Esta abordagem torna possível que o modelo capte, de forma simultânea (ao contrário das RNN), múltiplos tipos de dependências linguísticas, incluindo relações semânticas, sintáticas e contextuais em diversas escalas. Enquanto uma head foca-se na detecção de dependências locais entre palavras próximas, outra pode especializar-se na identificação de relações de longo alcance. Após o processamento individual de cada cabeça, os resultados são concatenados (**concat**) e projetados (**output projection**) novamente para o espaço vetorial original, produzindo uma representação contextual mais rica e expressiva.

A Multi-Head Attention foi uma das principais inovações introduzidas no artigo "Attention Is All You Need", contribuindo significativamente para a capacidade dos Transformers em modelar a linguagem de forma eficiente e paralelizável.

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^O$$
$$where head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

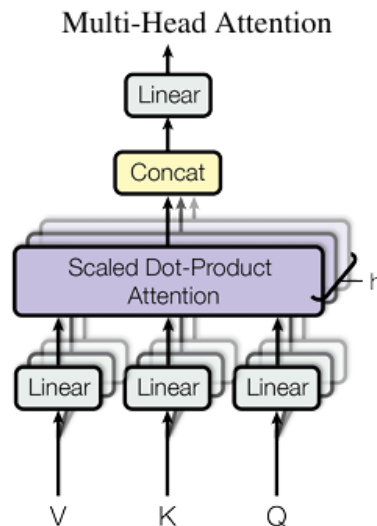


Figura 12 - Multi-Head Attention

2.2.6 Add & Norm

Os Transformers são construídos através do *concat* sucessivo de múltiplas camadas, permitindo ao modelo aprender representações progressivamente mais abstratas e complexas dos inputs.

O treino de Deep Neural Networks apresenta desafios, nomeadamente os problemas de *vanishing gradients* e *exploding gradients*, que podem comprometer a estabilidade e a eficiência do processo de aprendizagem. Para mitigar estes problemas, a arquitetura Transformer incorpora dois mecanismos fundamentais em cada subcamada, tanto nos blocos de Attention como nas Feedforward Neural Network.

Residual Connections (Add)

À medida que a informação percorre uma Rede Neural Profunda, as sucessivas transformações aplicadas em cada camada podem alterar significativamente as representações iniciais, conduzindo à perda de informação relevante. No decorrer do processo de backpropagation, os gradientes devem igualmente atravessar todas estas camadas, podendo tornar-se excessivamente diminutos (*vanishing gradients*) ou excessivamente amplos (*exploding gradients*), dificultando a atualização eficiente dos parâmetros.

Com o intuito de mitigar estas questões, foram introduzidas as **Residual Connections** [24]. Esta técnica estabelece uma trajetória direta que possibilita a informação e os gradientes a contornarem, de forma parcial, determinadas transformações da rede.

$$Output = x + Sublayer(x)$$

Esta abordagem super simples facilita a propagação de informação ao longo da rede, melhora o fluxo de gradientes durante o treino e contribui significativamente para a estabilidade de arquiteturas profundas.

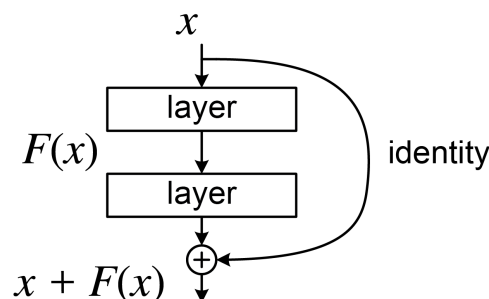


Figura 13 - Residual Connection

Layer Normalization (Norm)

A **Layer Normalization** (Layer Norm) é uma técnica usada para estabilizar o treino de Redes Neurais Profundas, garantindo que as ativações internas de cada camada preservam uma escala consistente ao longo do processo de aprendizagem. Em redes muito profundas, os valores intermédios podem apresentar valores excessivos ou insignificantes, dificultando a convergência do modelo e podendo conduzir à instabilidade durante o processo de treino [25].

Em termos simples, a Layer Norm garante que os valores internos da rede não apresentem uma variação excessiva, seja para o valor máximo ou para o valor mínimo.

Post-Layer Norm Transformer Design

Na arquitetura original apresentada no artigo "Attention Is All You Need", é adotado o esquema designado por **Post-Layer Normalization (Post-LN)**.

Neste formato, a operação Add & Norm é aplicada **após** cada subcamada do Transformer, incluindo o módulo de Attention e a Feedforward Neural Network. A transformação da subcamada é efetuada em primeiro lugar, sendo posteriormente adicionado o valor residual da entrada original (Add). A aplicação da Layer Normalization (Norm) é realizada em último lugar. Este design foi utilizado no Transformer original e desempenha um papel importante na estabilização do treino de Deep Neural Networks. Contudo, apesar da sua simplicidade, foi posteriormente verificado que o esquema Post-LN pode dificultar o fluxo de gradientes em modelos muito profundos, resultando em problemas de instabilidade durante o treino [26].

**Post Layer Normalization
(Post-LN)**

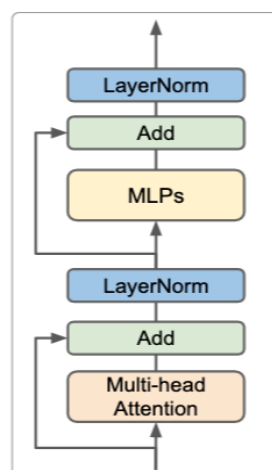


Figura 14 - Post-LN

Pre-Layer Norm Transformer Design

Pre-Layer Normalization (Pre-LN) constitui uma variante da arquitetura Transformer, desenvolvida posteriormente ao design original, com o objetivo principal de otimizar a estabilidade do treino em Redes Neurais Profundas.

Em contraste com o esquema Post-Layer Norm, em que a normalização é aplicada após a subcamada, no **Pre-LN** a Layer Normalization é aplicada antes da operação de Attention ou da Feedforward Neural Network. Deste modo, a entrada é primeiramente normalizada e, de seguida, processada pela subcamada. O resultado é, então, adicionado à representação original por meio de uma Residual Connection. Esta pequena modificação, à primeira vista simples, tem um impacto tremendo no comportamento do modelo. A normalização da entrada antes da transformação promove a estabilidade do fluxo de gradientes, reduzindo problemas de instabilidade em arquiteturas profundas e consequentemente facilitando o treino.

A abordagem **Pre-LN tornou-se a mais comum** em modelos modernos de linguagem generativa, incluindo arquiteturas do tipo GPT, especialmente em arquiteturas com elevado número de camadas.

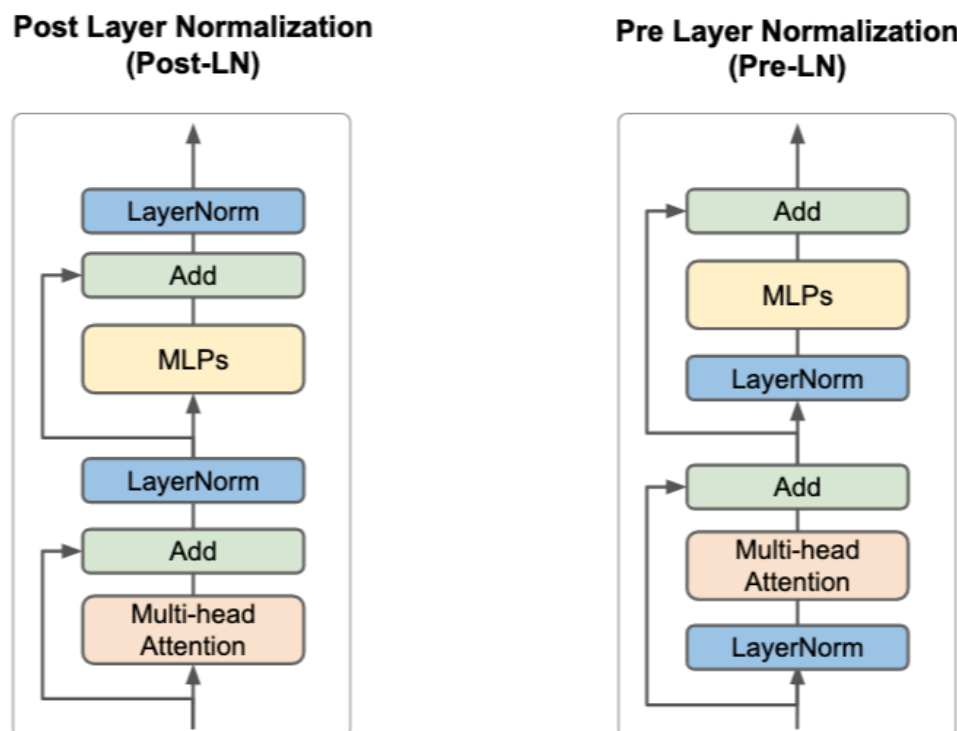


Figura 15 - Post LN vs Pre LN

2.2.7 Feedforward Neural Network

A Feedforward Neural Network (FFN) [27] é uma componente fundamental de cada bloco Transformer, aplicada após a Attention. A sua função primordial é introduzir **transformações não lineares** (forte das FFN) nas representações dos tokens, permitindo ao modelo aprender relações mais complexas para além das dependências capturadas.

O mecanismo de Attention é responsável pela modelação das interações entre diferentes tokens da sequência, enquanto a Feedforward Neural Network atua de forma independente em cada posição, refinando a representação de cada token individualmente. Desta forma, cada vetor é transformado de maneira **não linear**, aumentando a capacidade expressiva do modelo. **Em geral, esta rede apresenta uma arquitetura fully connected com duas camadas lineares, separadas por uma função de ativação não linear**, como a função ReLU ou a função GELU.

2.2.8 Language Modelling / Output Projection & Softmax

Após o processamento da sequência pelos blocos Transformer sucessivos, o modelo gera representações vetoriais contextualizadas para cada token. Todavia, as representações em questão não correspondem diretamente a palavras do vocabulário. Para converter estas representações numa previsão linguística, é aplicada uma projeção linear que transforma os vetores internos do modelo para um espaço com dimensão igual ao tamanho do vocabulário (B,T,V) [28]. O resultado desta operação consiste num conjunto de valores designados por **logits**, onde cada valor representa a relevância associada a um possível token do vocabulário [29].

Posteriormente, é aplicada a função softmax, responsável por converter os **logits** numa distribuição de probabilidades. Assim, o modelo atribui uma probabilidade a cada token possível, permitindo seleccionar o próximo elemento da sequência.

A função *softmax* é definida como:

$$\text{Softmax}(\hat{y}_i) = \frac{e^{\hat{y}_i}}{\sum e^{\hat{y}_j}}$$

Nos modelos autorregressivos, este processo é repetido iterativamente, permitindo a geração sequencial de texto token a token.

2.2.9 Treino

Os Transformers consistem em arquiteturas compostas por milhões, ou até milhares de milhões, de parâmetros treináveis. Para que o modelo aprenda padrões linguísticos a partir dos dados, é necessário executar um processo iterativo de treino, durante o qual os parâmetros da rede são progressivamente ajustados.

O objetivo do treino consiste em minimizar a discrepância entre as previsões produzidas pelo modelo e os valores esperados presentes nos dados de treino.

2.2.9.1 Gradientes e Backpropagation

No cálculo vetorial, o **gradiente** corresponde a um vetor que aponta na direção do maior aumento de uma grandeza escalar, como a temperatura ou a pressão. O módulo do vetor gradiente indica a taxa de variação da grandeza por unidade de distância [30].

O processo de aprendizagem dos Transformers baseia-se no **cálculo de gradientes**, que representam a direção e magnitude da variação da função de perda relativamente aos parâmetros do modelo. Após cada previsão realizada pelo Transformer, é calculado o erro entre a saída produzida e o valor esperado. Depois, usando o algoritmo de **backpropagation** este erro é propagado ao longo da rede, possibilitando a determinação da contribuição de cada parâmetro para o erro final [31].

É importante perceber que **backpropagation não altera pesos nenhuns**, calcula apenas o gradiente do erro em relação a cada parâmetro. O princípio de funcionamento é este:

Após o cálculo do erro, o mesmo percorre a rede inteira de trás para frente e calcula o gradiente (“Quanto o loss muda se eu mexer um pouco nesse peso”) da loss em relação a cada parâmetro.

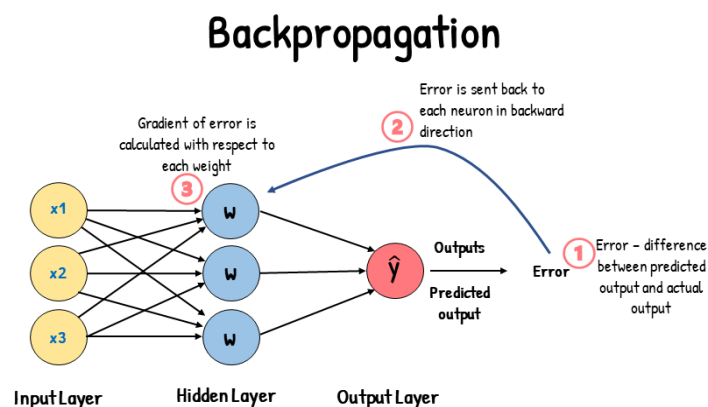


Figura 16 - Gradientes e Backpropagation

2.2.9.2 Loss Function

As **Loss Functions** são utilizadas para quantificar a discrepância entre as previsões do modelo e os valores reais presentes nos dados de treino. O propósito do processo de aprendizagem consiste em minimizar essa função ao longo das iterações de treino.

No âmbito das tarefas de modelação de linguagem, a função mais utilizada é a **Cross-Entropy Loss**, que quantifica a discrepância entre a distribuição de probabilidades prevista pelo modelo e a distribuição real esperada [32].

$$L = - \sum_{k=1}^K y_k \log(\hat{y}_k)$$

A **Cross-Entropy funciona como um treinador ideal**, avaliando não só se as previsões estão certas ou erradas, mas também o grau de confiança com que foram feitas:

- Certas e previsões confiantes? Penalização mínima
- Certas, mas previsões incertas? Penalização moderada
- Erradas, mas previsões incertas? Penalização moderada
- Erradas e previsões confiantes? Penalização severa

Isto cria o ambiente de aprendizagem perfeito! Só se deve ter confiança quando houver bons motivos para tal.

Exemplo:

Cenário 1: O e-mail é spam $Y = 1$, a IA tem 90% de confiança então $\hat{Y} = 0,9$. Perda = $-\log(0,9) \approx 0,105$ - **Uma pequena penalização por estar correto e confiante.**

Cenário 2: O e-mail é spam $Y = 1$, a IA tem apenas 10% de confiança então $\hat{Y} = 0,1$. Perda = $-\log(0,1) \approx 2,3$ - **Cerca de 22 vezes pior! Correto, mas demasiado incerto.**

Cenário 3: O e-mail não é spam $Y = 0$, a IA está 90% confiante de que é, então $\hat{Y} = 0,9$. Perda = $-\log(1-0,9) = -\log(0,1) \approx 2,3$ - **Penalização igualmente grave por estar confiantemente errado.**

Para além da *Cross-Entropy*, existem outras funções de perda utilizadas em diferentes contextos, incluindo *Mean Squared Error* (MSE), muito utilizada em modelos de Machine Learning.

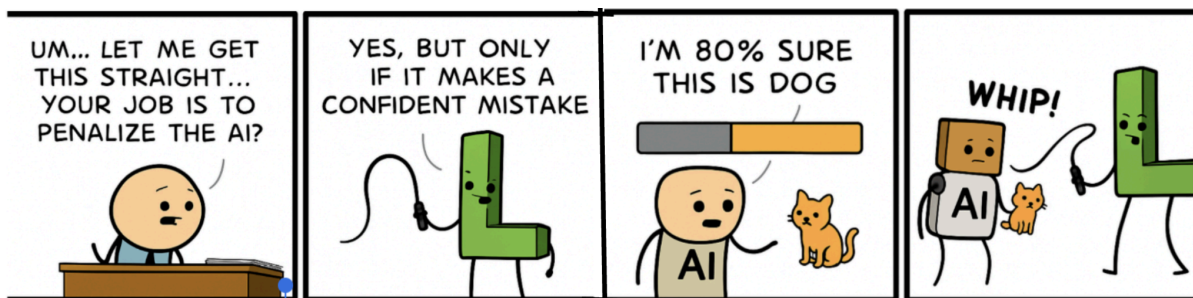


Figura 17 - Cross-Entropy

2.2.9.3 Otimizadores

Ao treinar uma Rede Neural, o nosso principal objetivo é minimizar a Loss Function, para tal, é necessário ajustar cuidadosamente os parâmetros do modelo, de modo a tornar as previsões da rede o mais precisas possível. Mas como é que se alteram exatamente esses parâmetros e em que medida? É aqui que entram em jogo os **Otimizadores** ou **Optimizers**.

Os **algoritmos de Otimização (Optimizers)** são responsáveis pela atualização dos parâmetros da rede, com base nos gradientes calculados durante o processo de backpropagation. No âmbito dos algoritmos mais utilizados, é de realçar o **AdamW**, que tem vindo a ser extensamente adotado no treino de Transformers modernos.

Outros métodos incluem Adam, RMSProp e variantes otimizadas para treino distribuído e modelos de larga escala.

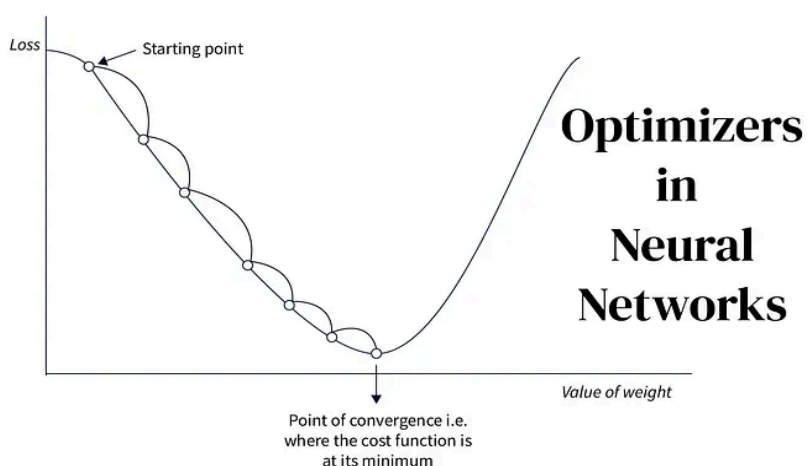


Figura 18 - Optimizers

2.2.10 Inferência

Após a fase de treino, o modelo pode ser utilizado em tarefas reais através do processo de inferência. Nesta etapa, os parâmetros adquiridos são mantidos constantes e o Transformer é usado para gerar previsões a partir de novos dados de entrada.

Nos modelos autorregressivos, a inferência é efetuada através da geração sequencial de texto **token a token**. A partir de um ponto de partida, o modelo calcula uma distribuição de probabilidades sobre o vocabulário e seleciona o próximo token mais provável. Este novo token é, então, adicionado à sequência de entrada, repetindo-se iterativamente o processo até que seja atingida uma condição de paragem.

A qualidade da geração não é determinada exclusivamente pelo treino do modelo, mas também pela estratégia adotada na seleção dos tokens durante o processo de inferência.

<i>Greedy Decoding</i>	<i>Beam Search</i>	<i>Top-k Sampling</i>	<i>Top-p Sampling</i>	<i>Temperature Sampling</i>
O modelo seleciona sempre o <i>token</i> com maior probabilidade em cada passo da geração.	Mantém múltiplas sequências candidatas simultaneamente durante a geração, explorando diferentes possibilidades antes de selecionar a sequência final mais provável.	O modelo restringe a seleção aos K <i>tokens</i> mais prováveis, realizando depois uma amostragem aleatória dentro desse subconjunto.	Seleciona dinamicamente o menor conjunto de <i>tokens</i> cuja probabilidade acumulada ultrapassa um determinado valor P.	Controla o grau de aleatoriedade da distribuição de probabilidades.
Simples Rápido Determinístico	Maior qualidade em tarefas estruturadas Muito usado em tradução automática	Mais criatividade Evita escolhas improváveis	Equilíbrio entre diversidade e coerência Mais adaptativo Amplamente usado em LLMs modernos	Normalmente combinada com <i>top-k</i> ou <i>top-p sampling</i> .
Texto repetitivo Pouca criatividade	Elevado custo computacional	Escolha fixa de K Pode eliminar tokens relevantes		

Tabela 4 - Estratégias de Inferência

2.3 Fine-Tuning

O Fine-Tuning ou estranhamente em Português Ajuste Fino, é a adaptação de um modelo pré-treinado a tarefas ou domínios específicos através de uma fase adicional de treino supervisionado. Em vez de treinar um modelo desde o início, utiliza-se um Transformer Pré Treinado, como os modelos da família GPT, aproveitando o conhecimento linguístico adquirido durante a fase de pré-treino. Este processo é realizado sobre conjuntos de dados significativamente menores e mais especializados, permitindo ajustar o comportamento do modelo para contextos particulares, como chatbot, programação, análise documental ou aplicações orientadas a um domínio [33]. **É como ter um motor e decidir a que carroçaria vou aplicar.**

O objetivo do Fine-Tuning consiste em transferir e especializar os padrões adquiridos durante o pré-treino, otimizando o desempenho em tarefas específicas sem a necessidade de repetir o processo de treino em larga escala, que é economicamente pesado. Esta abordagem possibilita uma significativa redução dos requisitos computacionais e da quantidade de dados necessários para obter modelos eficazes. Nos modelos contemporâneos, o Fine-Tuning desempenha um papel crucial, uma vez que a fase de pré-treino, por si só, não gera necessariamente ChatBots alinhados com instruções humanas. Esta adaptação só é alcançada através de etapas adicionais de especialização e alinhamento, que permitem a adaptação dos modelos a comportamentos desejados [34].

No âmbito do ecossistema open-source de Inteligência Artificial, o Fine-Tuning assume-se como o pilar mais forte. A fase de Pré-Treino de raiz exige infraestruturas computacionais extremamente dispendiosas, envolvendo milhares de GPUs, enormes volumes de dados e custos energéticos muito elevados. O desenvolvimento de modelos de raiz com milhares de milhões de parâmetros encontra-se, de modo geral, restrito a grandes organizações tecnológicas e centros de investigação. Desta forma, grande parte da comunidade open-source baseia-se na adaptação de modelos previamente treinados através de técnicas de Fine-Tuning. Esta abordagem possibilita a adaptação de modelos pré treinados às necessidades de qualquer utilizador.

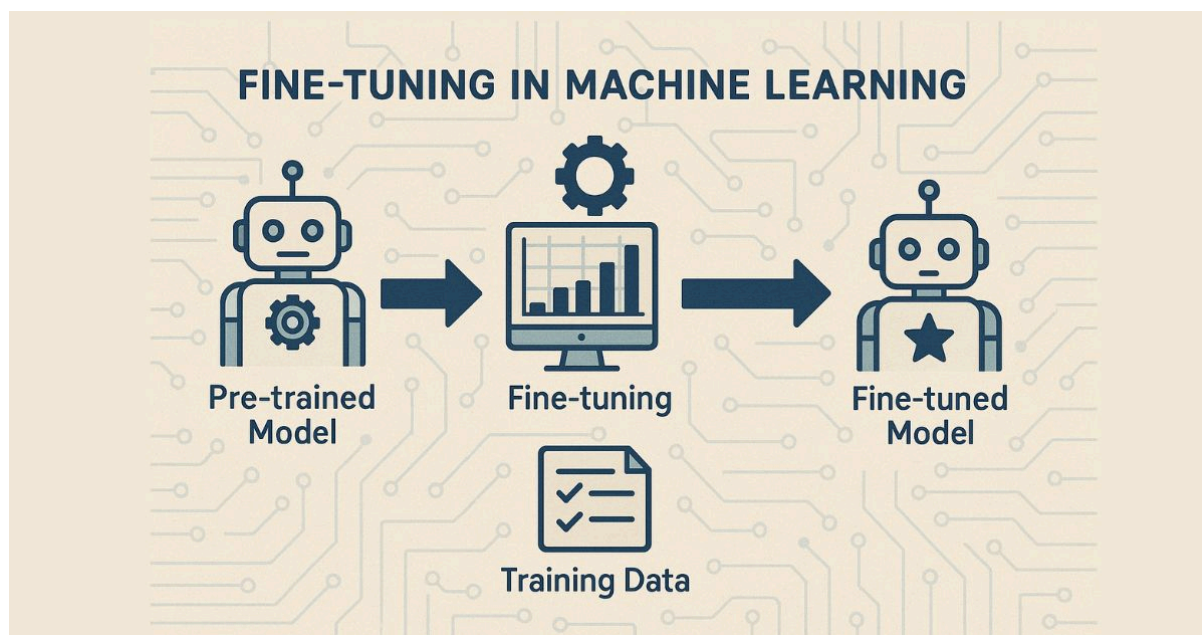


Figura 19 - Fine-Tuning

<i>Full Fine-Tuning</i>	<i>Parameter-Efficient Fine-Tuning (PEFT)</i>	<i>Instruction Fine-Tuning</i>	<i>Reinforcement Learning from Human Feedback (RLHF)</i>
Método tradicional, onde todos os parâmetros do modelo são atualizados durante o treino.	Abordagem moderna mais importante. Em vez de atualizar todos os pesos, apenas uma pequena parte do modelo é treinada.	Uma das formas mais importantes para LLMs modernos. O modelo é treinado em dados do tipo: Instrução → Resposta esperada Isto é o que transforma um modelo “bruto” num chatbot útil.	Técnica usada para alinhar modelos com preferências humanas. Humanos avaliam a resposta de um modelo.

Tabela 5 -Tipos de Fine-Tuning

2.4 Fine-Tuning vs Pre-Training

<i>Aspecto</i>	<i>Pre-Training</i>	<i>Fine-Tuning</i>
<i>Dados Necessários</i>	Quantidades enormes de dados não estruturados.	Quantidade pequena, foca-se em dados de uma área específica.
<i>Objetivo</i>	Construir um modelo capaz de produzir linguagem humana.	Especializar um modelo para uma tarefa específica.
<i>Processo</i>	Reunir dados, treinar o modelo, prever o próximo token.	Coletar dados de uma área específica, treinar de acordo com um novo dataset, gerar um output de acordo com a nova informação aprendida.
<i>Modificação do Modelo</i>	Modelo inteiro	Parte do modelo
<i>Custo Computacional</i>	Alto	Baixo
<i>Duração do Treino</i>	Semanas a meses	Dias
<i>Exemplos</i>	GPT	Chat-GPT

Tabela 6 - Pre-Training vs Fine-Tuning

3. Large Language Models

Um Large Language Model (LLM) é um modelo de inteligência artificial concebido para compreender e gerar linguagem natural. Estes modelos baseiam-se na arquitetura Transformer e são treinados em grandes volumes de dados textuais através de processos de aprendizagem supervisionada autónoma, tipicamente formulados como tarefas de previsão do próximo token, como já vimos [35].

Como já observámos, durante a fase de pré-treino, o modelo é sujeito a um processo de aprendizagem de padrões estatísticos da linguagem. Posteriormente, são aplicadas técnicas adicionais de adaptação, tais como o Fine-Tuning, incluindo abordagens como o ***Instruction Fine-Tuning*** e o ***Reinforcement Learning from Human Feedback (RLHF)***. Estas técnicas visam ajustar o comportamento do modelo para uma interação mais alinhada com instruções humanas e preferências de utilização.

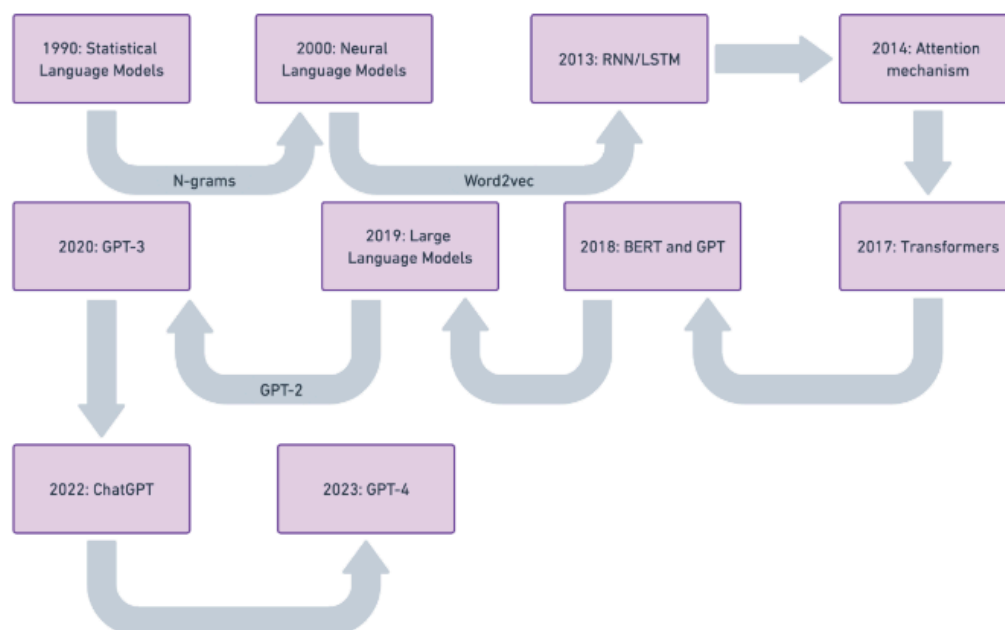


Figura 20 - Linha Cronológica dos LLM's

3.1 Scaling Laws

As Scaling Laws descrevem o comportamento dos modelos de Inteligência Artificial à medida que fatores como o **número de parâmetros**, a **quantidade de dados de treino** e o **poder computacional aumentam**. Estas leis miram a modelação matematicamente da relação entre a escala do modelo e o seu desempenho, possibilitando a previsão de como determinadas alterações influenciam a capacidade do sistema. Nos Large Language Models, de acordo com o paper “[Scaling Laws for Neural Language Models](#)” verificou-se empiricamente que o aumento da dimensão dos modelos, do volume de dados e da capacidade de processamento do computador utilizado durante o treino conduz frequentemente a melhorias consistentes em métricas como a perda (*loss*), a precisão e a capacidade de generalização.

As Scaling Laws exerceram uma influência notável no desenvolvimento de Large Language Models contemporâneos, mostrando que o incremento coordenado de parâmetros, dados e capacidade computacional frequentemente resulta na emergência de competências não observadas em modelos mais reduzidos. Esta ideia esteve na base do desenvolvimento de modelos de larga escala, incluindo sistemas da família GPT, e influenciou fortemente a evolução recente da Inteligência Artificial generativa.

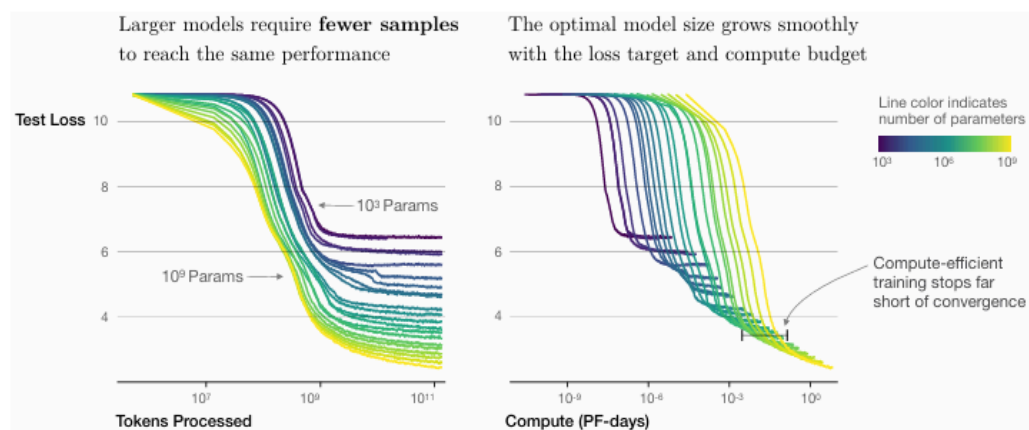


Figura 21 - [Scaling Laws for Neural Language Models](#)

3.2 Capacidades Emergentes em Large Language Models

Emergent Abilities referem-se a comportamentos e capacidades que não são observados (ou são observados de forma muito diminuta) em modelos pequenos, mas que surgem de forma evidente quando o modelo atinge uma certa escala (mais parâmetros, mais dados, mais processamento).

Como o paper original, “[Emergent Abilities of Large Language Model](#)” indica:

An ability is emergent if it is not present in smaller models but is present in larger models.

A ideia principal é que o aumento de escala não melhora apenas a "qualidade" de forma gradual. Em determinadas circunstâncias, **emergem novas competências que se distinguem por uma qualidade distinta**.

Vamos aprofundar com um exemplo:

Tarefa	Modelo Pequeno	Modelo Médio	Modelo Grande
Completar texto simples	✓	✓	✓
Responder a perguntas difíceis	✗	✓	✓
Raciocínio Multi-Step	✗	✗	✓

Tabela 7 - Emergent Abilities

Pensando de repente, esta conduta pode aparentar ser intuitiva: modelos com um número reduzido de parâmetros exibem uma capacidade intrínseca de representação limitada, ao passo que modelos de maior envergadura evidenciam desempenhos substancialmente superiores.

Contudo, o que o paper mostra, é que a partir de um limiar específico de escala, os modelos começam a demonstrar competências qualitativamente distintas, tais como a resolução de problemas que exigem raciocínio multi-step, a generalização para tarefas não observadas durante o período de treino e uma maior consistência na geração de código funcional.

A razão precisa para o surgimento destas capacidades emergentes permanece pouco clara.

De algumas das hipóteses discutidas, destaca-se o facto de os modelos de maior dimensão possuírem uma maior capacidade de representação interna, permitindo a codificação de estruturas linguísticas e semânticas mais complexas. Estes modelos demonstram também uma superior eficácia na captura de padrões estatísticos de elevada complexidade, presentes nos dados de treino, bem como na modelação de interações não-lineares profundas dentro da arquitetura da Rede Neural.

3.3 Limitações dos Large Language Models

Não obstante os avanços substanciais alcançados pelos Large Language Models, estes sistemas exibem ainda múltiplas limitações fundamentais que afetam a sua fiabilidade, interpretabilidade e aplicabilidade.

3.3.1 Alucinações (Hallucinations)

Uma das limitações mais difundidas dos LLMs consiste na propensão para a geração de informação incorreta ou inexistente, embora apresentada de forma plausível. Este fenómeno deve-se ao facto de o modelo não possuir uma noção explícita de verdade, mas sim uma modelação estatística da linguagem. Deste modo, mesmo na ausência de conhecimento sobre uma resposta, o modelo tem a capacidade de gerar conteúdo com elevada confiança linguística, embora careça de fundamentação factual [36].

3.3.2 Limitação de Contexto (Context Window)

A *Context Window* (**Context Window = Sequence Length**) de um Large Language Model (LLM) refere-se ao número máximo de tokens que o modelo é capaz de processar em simultâneo numa única entrada. Este limite estabelece a extensão da informação contextual que o modelo é capaz de considerar em cada etapa do processo de inferência, afetando diretamente a sua aptidão para compreender e produzir texto coerente em tarefas mais complexas. Uma context window mais extensa faculta ao modelo a capacidade de lidar com sequências mais extensas, aprimorando a manutenção da coerência em documentos extensos ou conversas prolongadas [37].

Em termos comportamentais, modelos como o ChatGPT podem sustentar conversas contínuas, mas não possuem memória persistente entre interações. Estes sistemas são comumente descritos como "**stateless**", no sentido em que não armazenam informação de sessões anteriores de forma permanente. Esta situação implica que o modelo apenas "tem acesso" ao conteúdo presente na sua janela de contexto atual. A informação anterior que

ultrapasse esse limite é excluída do cálculo, o que pode dar a impressão de que o modelo "não se lembra" de interações passadas.

3.3.3 Data Dependence e Custos Computacionais

Durante o Pre-Training, estes modelos são expostos a enormes corpora de texto provenientes de múltiplas fontes, frequentemente na ordem de centenas de gigabytes ou mesmo terabytes de dados. Esta necessidade nasce da observação de que os modelos aprendem padrões estatísticos da linguagem de forma puramente **data-driven**, sem regras explícitas [38].

O treino de um LLM implica um consumo computacional extremamente elevado, uma vez que requer o processamento de grandes volumes de dados através de arquiteturas com milhares de milhões de parâmetros. Este processo é habitualmente executado em infraestruturas compostas por milhares de GPUs ou TPUs, podendo prolongar-se durante semanas ou mesmo meses de computação contínua. Este custo elevado está associado ao facto de cada atualização dos parâmetros da rede requerer a execução de backpropagation em todas as camadas do modelo, sobre conjuntos de dados extremamente extensos, resultando também num consumo energético significativo [39].

O treino de modelos de raiz torna-se praticamente inacessível fora de grandes organizações tecnológicas ou laboratórios de investigação com elevado poder computacional.

Mesmo após terminar a fase de Pre-Training, os modelos demonstram custos computacionais elevados durante a fase de inferência. A geração de cada token requer múltiplas operações de Attention e processamento em toda a arquitetura, sendo que este custo aumenta proporcionalmente ao tamanho da context window e ao comprimento da sequência gerada. Em interações de longa duração ou em aplicações em tempo real, este fator pode adquirir uma relevância particular, afetando a latência e a eficiência do sistema. A presente dependência de grandes quantidades de dados e de recursos computacionais acarreta várias implicações práticas.

Claramente, estabelece uma barreira de entrada significativa para novos investigadores e organizações de menor dimensão. Em contrapartida, contribui para a centralização do desenvolvimento destes modelos em grandes empresas com capacidade infraestrutural adequada. Em resposta a estas limitações, têm emergido técnicas de otimização, tais como *Parameter-Efficient Fine-Tuning*, *Quantization*, *Distillation* e abordagens como o *Retrieval-Augmented Generation (RAG)*, que têm como objetivo reduzir o custo associado ao treino e à utilização destes sistemas.

3.4 Large Language Models como Foundation Models

Os Large Language Models não são tipicamente desenvolvidos para uma tarefa específica, mas sim para aprender representações gerais da linguagem, que posteriormente podem ser adaptadas a diferentes contextos e aplicações.

Em virtude da sua natureza generalista, os LLMs atuam como o núcleo de um ecossistema amplo de técnicas de adaptação e especialização. Em vez de serem sujeitos a um processo de formação novo para cada tarefa, são utilizados como ponto de partida para a aplicação de diversos métodos com vista à otimização ou orientação do seu comportamento. Dentro destas abordagens, destaca-se o **Fine-Tuning**, que permite ajustar os parâmetros do modelo para tarefas específicas, e o **Parameter-Efficient Fine-Tuning (PEFT)**, que otimiza este processo atualizando apenas uma pequena parte dos parâmetros. Técnicas como o **Retrieval-Augmented Generation (RAG)** possibilitam a expansão das capacidades do modelo através da incorporação de informação externa, sem que seja necessária qualquer modificação na sua estrutura interna. O comportamento dos LLMs pode também ser influenciado através do **Prompting Engineering**, um processo no qual instruções em linguagem natural são usadas para guiar diretamente a geração de texto [40].

Neste sentido, os LLMs podem ser entendidos como o ponto central de um ecossistema composto por diferentes camadas de adaptação: métodos que modificam o modelo (fine-tuning e PEFT), métodos que expandem o seu conhecimento (RAG) e métodos que controlam o seu comportamento em tempo de execução (Prompt Engineering).

Assim, fica evidenciado o papel dos LLMs como uma infraestrutura fundamental sobre a qual se erguem os sistemas modernos de inteligência artificial generativa.



Figura 22 - Dimensão dos LLM's

3.5 Avaliação de Large Language Models - Benchmarks

No momento em que as empresas apresentam novos Large Language Models, a avaliação do desempenho não se fundamenta exclusivamente em métricas internas de treino. Em alternativa, são utilizados conjuntos de testes padronizados, denominados "**benchmarks**", que permitem comparar diferentes modelos de forma consistente e objetiva em várias capacidades linguísticas e cognitivas.

A avaliação do modelo em tarefas específicas, tais como o conhecimento geral, o raciocínio lógico, a matemática ou a programação, é efetuada por meio da aplicação de benchmarks. Por exemplo, o **MMLU** benchmark é amplamente utilizado para medir o conhecimento geral em múltiplas áreas do saber, recorrendo normalmente a questões de escolha múltipla. Já o **GSM8K** benchmark avalia a capacidade de resolução de problemas matemáticos que exigem raciocínio passo a passo, sendo particularmente relevante para testar competências de lógica.

Em muitos casos, as atuais avaliações incluem não só testes automáticos, mas também julgamentos humanos.

Um excelente website recomendado para analisar benchmarks de modelos é o [LLM Leaderboard 2026 — Compare 228 AI Models Across 186 Benchmarks | BenchLM.ai](#).

4. Técnicas de Fine-Tuning

4.1 Fine-Tuning para Tarefas Específicas

Adapta os grandes modelos de linguagem a tarefas específicas, utilizando dados devidamente formatados e limpos.

<i>Tarefa</i>	<i>Descrição</i>	<i>Modelos</i>
Sumarizar texto	Condensar textos longos em resumos coerentes, mantendo as informações essenciais.	GPT-3, T5
Gerar código	Geração automática de código com base em descrições em linguagem natural.	Codex, GPT-3
Classificar	Classificar texto em categorias predefinidas	BERT, RoBERTa

Tabela 8 - Exemplos de Fine-Tuning para Tarefas Específicas

4.2 Fine-Tuning para Domínios Específicos

<i>Domínio</i>	<i>Modelo Base</i>	<i>Descrição</i>
Saúde	PaLM 2	O Med-PaLM 2 foi treinado com conjuntos de dados médicos cuidadosamente selecionados e é capaz de responder com precisão a perguntas médicas, alcançando um desempenho comparável ao dos profissionais de saúde.
Finanças	LlaMA	O FinGPT , é um modelo concebido especificamente para o setor financeiro, melhora a investigação e a cooperação financeiras, promovendo a acessibilidade dos dados e abordando questões específicas do setor financeiro.
Direito	Alpaca Plus 7B	O LAWGPT , concebido especificamente para aplicações jurídicas na China, demonstra uma capacidade superior no tratamento de tarefas jurídicas.

Tabela 9 - Exemplos de Fine-Tuning para Domínios Específicas

4.3 Técnicas de Parameter-Efficient Fine-Tuning (PEFT)

Em vez de realizar a atualização completa de milhares de milhões de parâmetros, estas abordagens introduzem componentes adicionais treináveis ou modificam apenas partes específicas da arquitetura. Em consequência, os custos computacionais e de armazenamento são significativamente reduzidos, tornando o processo de adaptação mais eficiente e acessível. Para além da eficiência computacional, os métodos PEFT contribuem igualmente para a mitigação do problema do esquecimento catastrófico (*catastrophic forgetting*), um fenómeno em que as Redes Neurais perdem parcialmente o conhecimento previamente adquirido quando são sujeitas a treino em novos conjuntos de dados ou tarefas específicas [41].

Em cenários onde existem uma quantidade limitada de dados de treino, os métodos PEFT demonstram um desempenho comparável e, por vezes, superior ao Full Fine-Tuning, apresentando ainda uma melhor capacidade de generalização para contextos fora do domínio de treino. Estas abordagens têm sido extensivamente aplicadas em tarefas especializadas.

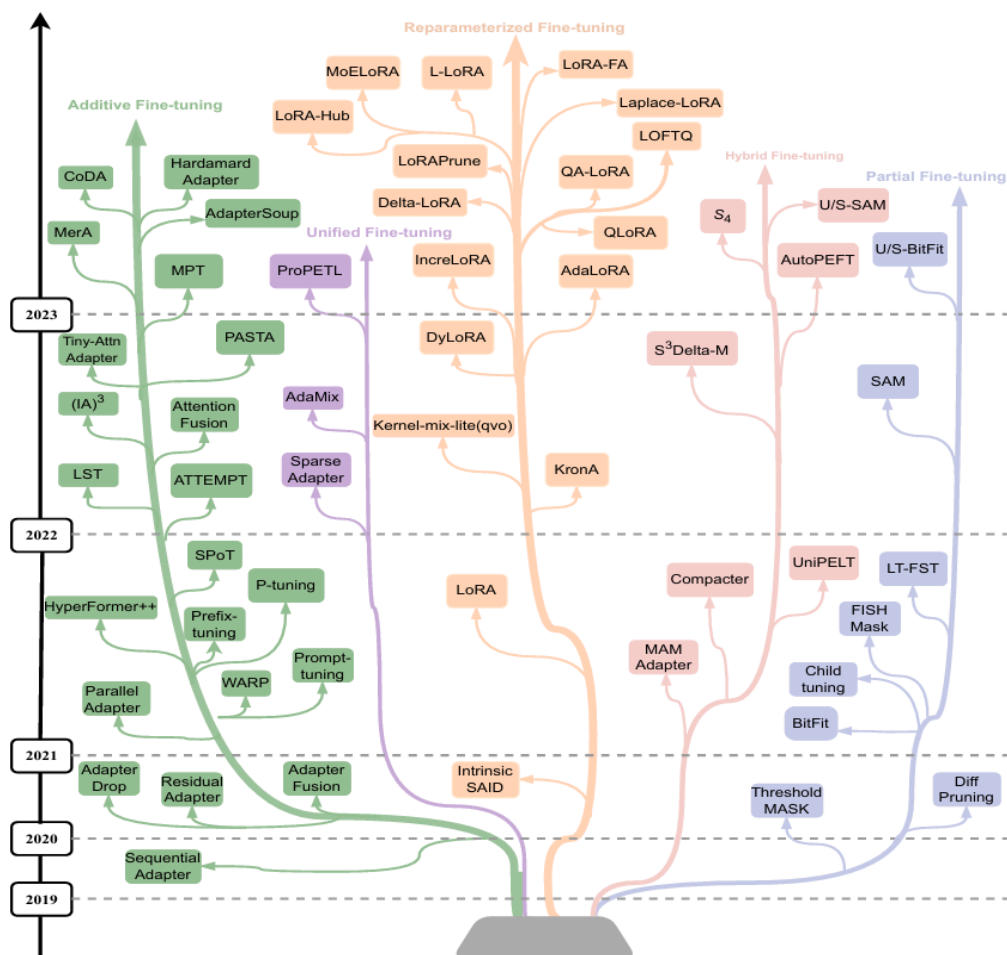


Figura 23 - Desenvolvimento dos métodos PEFT nos últimos anos

4.3.1 Adapters

Os [Adapters](#) constituem uma técnica de **Parameter-Efficient Fine-Tuning** fundamentada na inserção de reduzidas camadas treináveis no âmbito da arquitetura Transformer. Em vez de se proceder à atualização de todos os parâmetros do modelo, os pesos originais permanecem congelados, sendo apenas os módulos adicionais sujeitos a um processo de treino. Em circunstâncias convencionais, os adapters são incorporados subsequentemente às camadas de Attention ou às Feedforward Neural Network do Transformer [42].

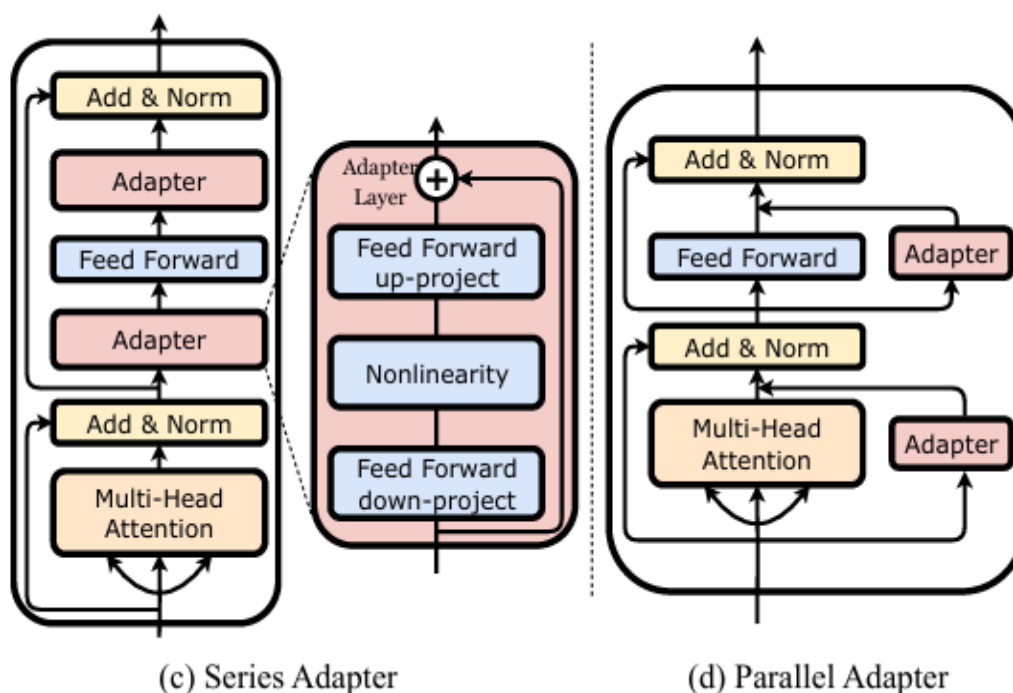


Figura 24 - Tipos de Adapter

O modelo pré-treinado permanece praticamente inalterado. A formação de pequenos blocos neurais adicionais entre camadas é um processo contínuo, no qual apenas os parâmetros desses blocos são sujeitos a atualizações durante o período de treino. A arquitetura típica consiste na projeção para uma dimensão inferior (**down-projection**), na **ativação não linear** e na projeção de volta para a dimensão original (**up-projection**).

<i>Vantagens</i>	<i>Desvantagens</i>
Muito eficiente em termos de memória	Introduz módulos adicionais na arquitetura
Reduz armazenamento necessário	Pode aumentar a latência de inferência
Permite reutilizar o mesmo modelo base	Em algumas tarefas pode ter desempenho inferior ao Full Fine-Tuning

Tabela 10 - Vantagens e Desvantagens dos Adapters

4.3.2 LoRA (Low-Rank Adoption)

Low-Rank Adaptation é uma técnica de Parameter-Efficient Fine-Tuning (PEFT), que permite a adaptação de um modelo sem a necessidade de atualização simultânea de todos os seus parâmetros. Em vez de proceder à modificação direta dos pesos originais do Transformer, o **LoRA** introduz pequenas matrizes treináveis de baixo rank nas camadas lineares do modelo, especialmente nas projeções associadas ao mecanismo de Attention.

Durante o *fine-tuning*, os pesos originais do modelo permanecem congelados.

O LoRA assume que:

As alterações necessárias para adaptar o modelo podem ser representadas por matrizes muito menores.

Assim, em vez de atualizar uma matriz completa W , o modelo aprende apenas uma aproximação:

$$W' = W + BA$$

W = Representa os pesos originais congelados, B e A = Matrizes pequenas treináveis, BA = Adaptação introduzida pelo LoRA

O funcionamento do LoRA baseia-se no facto de o modelo pré-treinado permanecer completamente congelado durante o processo de adaptação, ou seja, os seus parâmetros originais não são atualizados. São introduzidas pequenas matrizes adicionais nas camadas lineares do modelo, que passam a ser os únicos componentes sujeitos a treino. Estas matrizes aprendem a representar as adaptações necessárias para a nova tarefa, sem modificar diretamente o conhecimento já adquirido pelo modelo base. Durante a fase de inferência, as atualizações adquiridas podem ser combinadas com os pesos originais do modelo, permitindo que o sistema funcione como uma única entidade coerente, mas com o comportamento ajustado ao domínio específico para o qual foi treinado [43].

LoRA é frequentemente aplicado nas projeções lineares do mecanismo de Attention dos Transformers, particularmente nas matrizes associadas ao Query (Q), Key (K), Value (V) e à Output Projection. É precisamente nestes pontos que as alterações introduzidas pelo LoRA têm maior impacto na forma como o modelo processa e relaciona a informação entre tokens.

Low Rank Adaptation (LoRA) Overview

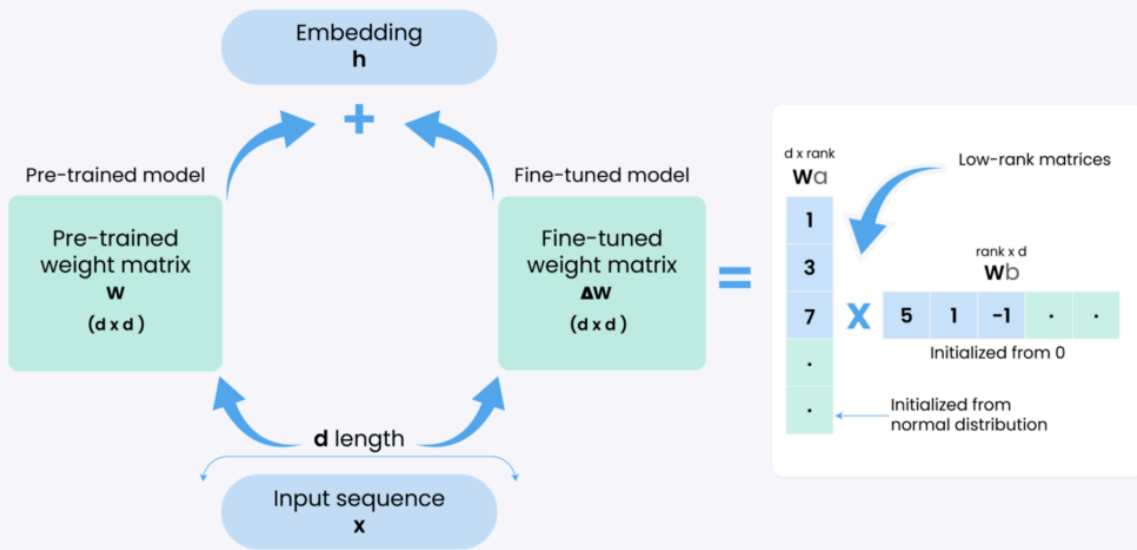


Figura 25 - LoRA

5. Prompt Engineering

Prompt Engineering pode ser entendido como o processo de conceber e otimizar instruções de entrada (**prompts**) de forma a orientar o comportamento de modelos de linguagem para produzir respostas específicas e consistentes. Esta conceção pode ser interpretada como uma interface entre a intenção do utilizador e a capacidade de geração do modelo. Consiste na criação e melhoria de instruções, perguntas ou contextos fornecidos ao modelo, com o objetivo de maximizar a relevância e a precisão das respostas. Dado que estes modelos são treinados em grandes volumes de dados e não possuem uma compreensão explícita das intenções humanas, a formulação adequada do prompt torna-se um fator crítico para o seu desempenho [44]. É um tema bastante abordado no mundo dos LLM 's, pois é um método totalmente gratuito, onde a arte está na maneira como pedimos ao modelo alguma informação.

5.1 Zero-Shot Prompting e Few-Shot Prompting

No domínio do Prompt Engineering, o **Zero-Shot Prompting** corresponde à capacidade de um Large Language Model executar uma tarefa sem necessidade de receber exemplos prévios dessa tarefa no prompt. O modelo em questão fundamenta-se unicamente no conhecimento adquirido durante o período pré-treino e na instrução fornecida pelo utilizador.

Exemplo:

Classifica o sentimento da seguinte frase como positivo ou negativo:
"Este filme foi fantástico."

Apesar de não serem fornecidos exemplos, modelos modernos conseguem executar corretamente a tarefa.

Few-Shot Prompting consiste em fornecer ao modelo alguns exemplos da tarefa antes da instrução principal. Estes exemplos funcionam como demonstrações do comportamento esperado, ajudando o modelo a compreender melhor o padrão de resposta pretendido.

Exemplo:

Texto: "Adorei este produto."
Sentimento: Positivo

Texto: "O filme foi interessante."
Sentimento:

Neste caso, o modelo infere o padrão apresentado nos exemplos e tenta continuar a sequência de forma coerente.

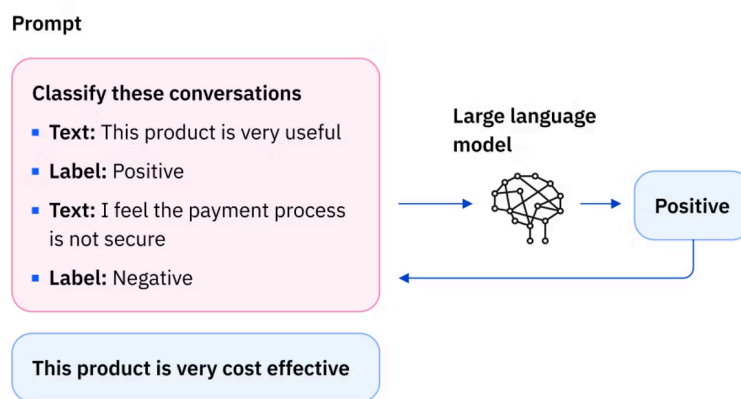


Figura 26 - Few-Shot Prompting

5.2 Chain-of-Thought (CoT)

Prompt Engineering integra uma técnica de extrema importância denominada **Chain-of-Thought (CoT)**, que consiste em incentivar o modelo a executar um raciocínio intermédio passo a passo antes de alcançar a resposta final. O modelo é orientado para decompor o problema em etapas intermédias de raciocínio, simulando um processo lógico mais estruturado.

Vamos observar alguns tipos de CoT:

<i>Zero-Shot CoT</i>	<i>Few-Shot CoT</i>
Basta adicionar frases como: “vamos pensar passo a passo”	É necessário fornecer exemplos completos com raciocínio intermédio

Tabela 11 - How To Zero-Shot CoT e Few-Shot CoT

5.3 Role Prompting

Role Prompting consiste em atribuir explicitamente um papel ou identidade ao modelo através do prompt, de forma a condicionar o estilo, a estrutura e o comportamento da resposta gerada.

Exemplo:

Prompt Normal → “Explica o que é uma Rede Neural.”

Role Prompting → “Explica o que é uma Rede Neural como se fosse um professor universitário de Inteligência Artificial.”

Com a utilização deste método o nível de detalhe tende a aumentar, tornando-se mais técnico ou mais simples, enquanto o vocabulário adquire um caráter mais formal, educativo ou técnico.

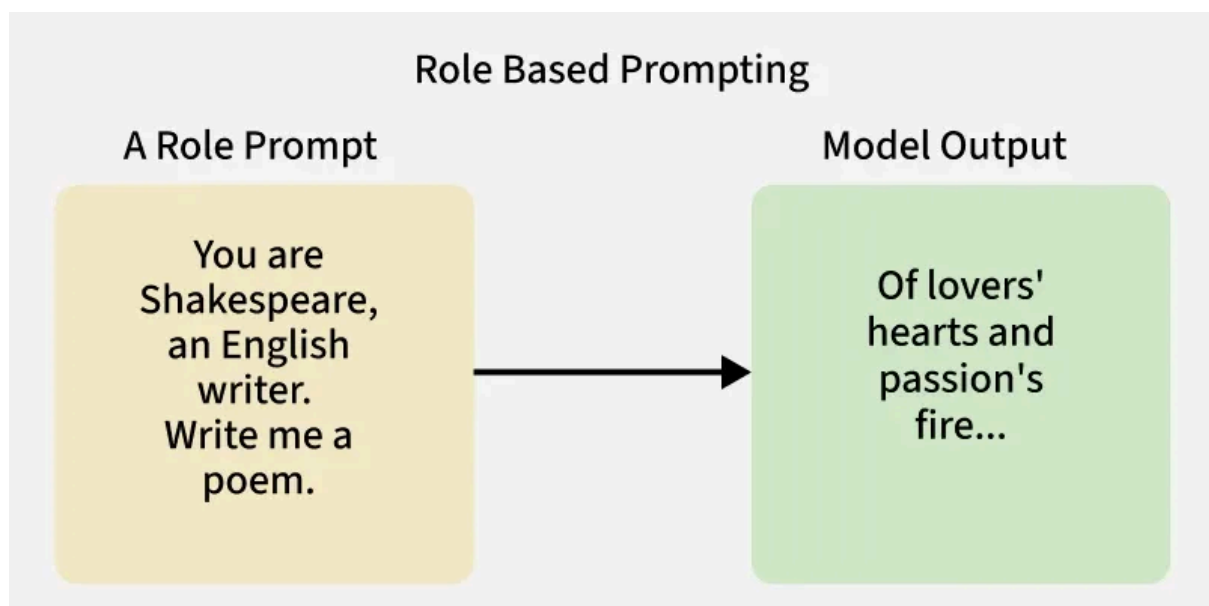


Figura 27 - Role Based Prompting

6. Retrieval Augmented Generation (RAG)

Retrieval Augmented Generation é uma técnica utilizada para expandir as capacidades de Large Language Models, combinando o conhecimento interno do modelo com fontes

externas de informação. Em vez de depender exclusivamente dos dados aprendidos durante o treino, o modelo passa a consultar sistemas de informação antes de gerar uma resposta. Este processo permite o acesso a informação atualizada, especializada ou altamente específica, sem necessidade de re-treinar o modelo base [44].

O funcionamento do RAG pode ser descrito como a interação entre dois componentes principais: o módulo de **retrieval** e o módulo de **generation**. A componente de recuperação é responsável por pesquisar e selecionar documentos relevantes a partir de bases de dados, repositórios de texto ou conhecimento externo estruturado. Estes documentos são então fornecidos ao modelo de linguagem como contexto adicional. A componente de geração utiliza tanto o input do utilizador como a informação recuperada para produzir uma resposta final mais informada, coerente e contextualizada. Com a aplicação de RAG, o modelo não depende apenas da sua memória paramétrica, mas também de conhecimento externo dinâmico.

<i>Limitações das LLM que RAG mitiga</i>	<i>Vantagens do RAG</i>
Quando o modelo não possui uma resposta adequada, ele inventa informações, aquilo que chamamos de Hallucinations.	Treinar novamente um LLM para cada nova atualização de conhecimento é um processo caro e demorado. RAG elimina essa necessidade ao integrar dados externos.
Como os LLMs têm uma data de corte em relação ao treino, não conseguem acessar eventos ou mudanças recentes, o que reduz a sua utilidade em diversos contextos.	Conectar o modelo a fontes de dados em tempo real permite que o mesmo forneça respostas baseadas nos acontecimentos mais recentes.
Termos semelhantes, mas com significados distintos em diferentes contextos, podem levar o modelo a gerar respostas imprecisas.	Os desenvolvedores podem ajustar as fontes de conhecimento utilizadas pelo modelo, garantindo que esteja alinhado aos objetivos da organização e evitando a exposição de informações sensíveis

Tabela 12 - RAG - Benefícios e limitações que mitiga

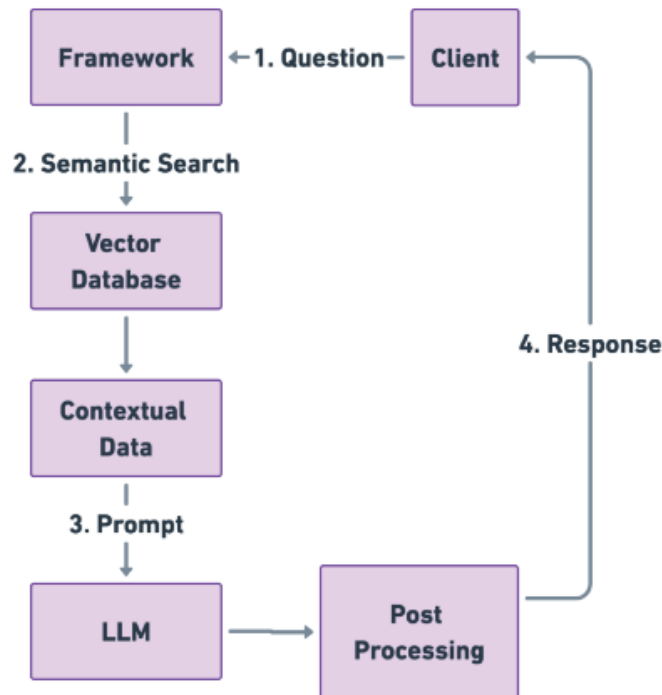


Figura 28 - RAG

6.1 How To RAG

O sistema recebe o input do utilizador, correspondendo à questão ou solicitação inicial. Este input é, subsequentemente, convertido numa representação vetorial através de um processo de **embedding** (cá estão eles de novo), que permite a captura do seu significado semântico num espaço de alta dimensão.

É executada a etapa de recuperação, na qual o sistema pesquisa documentos relevantes numa base de dados externa, geralmente por meio de métricas de similaridade entre embeddings. Após a pesquisa inicial, os documentos recuperados são submetidos a uma fase de ordenação, onde são classificados de acordo com a sua relevância em relação à consulta original, garantindo que apenas a informação mais pertinente seja utilizada.

De seguida, é construída uma **context window** que combina a consulta do utilizador com os documentos seleccionados. Este contexto enriquecido é, por conseguinte, fornecido ao modelo de linguagem, que realiza a etapa final de geração. Por fim, o modelo gera uma resposta que incorpora tanto o conhecimento interno adquirido durante o período de treino como a informação externa recuperada [45].

RAG deve ser compreendido como um pipeline completo de processamento de informação, onde cada etapa desempenha um papel fundamental na melhoria da precisão, relevância e atualidade das respostas geradas.

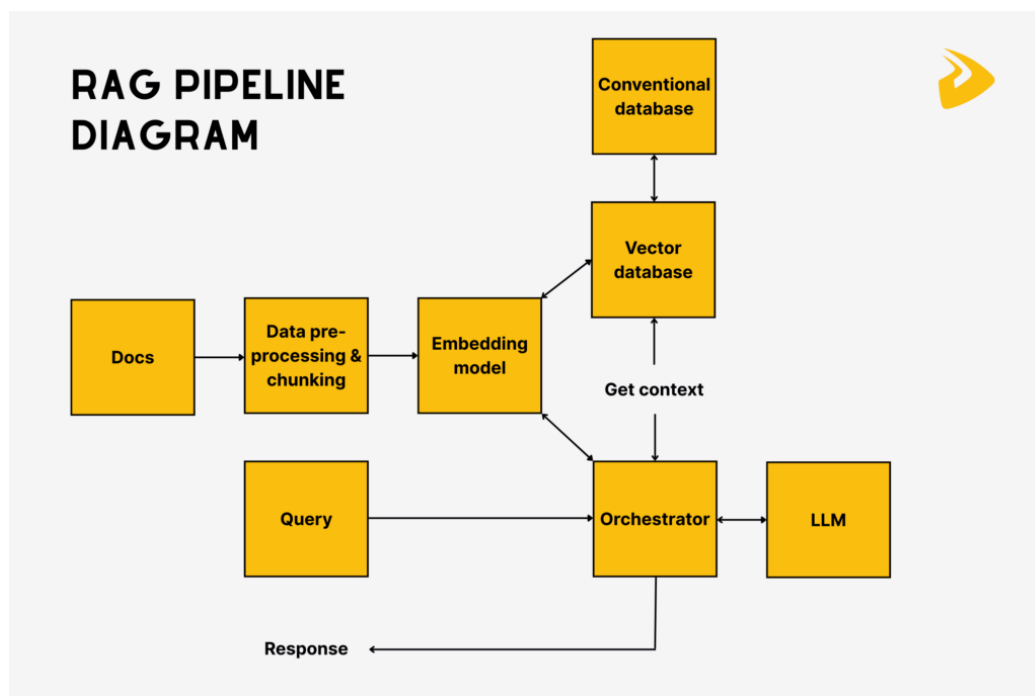


Figura 29 - RAG Step-by-Step

6.2 RAG vs Fine-Tuning

Ambas as abordagens visam otimizar o desempenho dos Large Language Models, embora através de mecanismos substancialmente distintos.

	<i>RAG</i>	<i>Fine-Tuning</i>
<i>Natureza da Tarefa</i>	Particularmente relevante em contextos profissionais que requeiram a compreensão de diversos contextos e a integração de conhecimentos externos.	Adequado para tarefas em que a adaptação a padrões específicos dentro de um domínio é crucial.
<i>Disponibilidade dos Dados</i>	Requer uma base de conhecimento para uma pesquisa eficaz, o que pode limitar a sua aplicabilidade em domínios com pouca informação externa.	Adaptável a cenários com dados específicos da tarefa limitados, tirando partido do conhecimento pré-existente durante a fase de pré-treino.
<i>Custo Computacional</i>	Muito exigente em termos computacionais, especialmente durante o processo de recuperação, o que pode afetar as aplicações em tempo real.	Geralmente menos exigente em termos computacionais, tornando-o mais adequado para aplicações com requisitos rigorosos de latência.
<i>Diversidade do Output</i>	Destaca-se na geração de resultados diversificados e contextualmente relevantes, graças ao seu mecanismo de recuperação de conhecimento.	Só conseguem adaptar-se eficazmente a domínios específicos durante o treino, e é necessário realizar um novo treino geral para que funcionem em novos domínios.
<i>Fonte de Conhecimento</i>	Depende inteiramente de fontes de conhecimento externas, o que pode introduzir imprecisões, dependendo da qualidade da informação obtida.	Não pode ser tendencioso, mas está limitado ao conhecimento adquirido durante o pré-treino, o que pode suscitar dificuldades na adaptação a domínios totalmente novos ou de nicho.

<i>Casos de Uso</i>	Ideal para tarefas que beneficiam de uma combinação de capacidades generativas e acesso a informação externa, como os chatbots no apoio ao cliente.	Eficaz para aplicações específicas de cada domínio, como a análise de documentos na área da saúde.
<i>Complexidade do Treino</i>	Implica um treino conjunto dos componentes de geração e de recuperação, o que aumenta a complexidade do processo de treino.	Implica procedimentos de treino mais simples, especialmente quando se utilizam modelos pré-treinados com conjuntos de dados específicos para cada tarefa e facilmente acessíveis.

Tabela 13 - RAG vs Fine-Tuning

7. Model Compression

[Model Compression](#) constitui uma técnica onde o objetivo é a redução do tamanho dos modelos sujeitos a treino. Os Large Language Models podem atingir uma elevada precisão, mas sempre associada a um custo elevado em termos de recursos. As técnicas de compressão visam comprimir os modelos sem que ocorra uma redução significativa do desempenho.

7.1 Quantization

A **Quantization** é uma técnica utilizada para reduzir o tamanho e os requisitos computacionais dos LLM 's, através da representação dos seus pesos e ativações com uma precisão numérica inferior. Em vez de utilizar formatos de alta precisão, como 32 bits em vírgula flutuante (float), os parâmetros do modelo são convertidos para representações mais compactas, como 16, 8 ou até 4 bits. Esta abordagem é importante para a implementação eficiente de LLMs em ambientes com recursos limitados, permitindo reduzir significativamente o consumo de memória e acelerar o processo de inferência, com uma perda de desempenho geralmente controlada [46].

<i>INT8 Quantization</i>	<i>INT4 Quantization</i>	<i>FP16/BP16</i>
Representação dos pesos a 8 bits, sendo um dos métodos mais comuns devido ao equilíbrio entre eficiência e desempenho.	Representação dos pesos a 4 bits, permitindo uma redução significativa de memória, mas com maior risco de perda de qualidade.	Formatos intermédios de float, frequentemente utilizados como compromisso entre precisão e eficiência.

Tabela 14 - Quantization Níveis de Precisão

7.2 Pruning

Consiste na remoção seletiva de parâmetros de uma Rede Neural, com o propósito de reduzir o seu tamanho e o seu custo computacional sem afetar significativamente o seu desempenho. A ideia do **Pruning** assenta na premissa de que, durante o treino de LLM 's, uma proporção significativa dos pesos tende a ter um impacto reduzido na saída final. Deste modo, em vez de manter todos os parâmetros, é possível eliminar aqueles considerados menos relevantes, tornando o modelo mais eficiente.

<i>Unstructured Pruning</i>	<i>Structured Pruning</i>
São removidos pesos individuais da rede, independentemente da sua posição na arquitetura. Ou seja, alguns valores dentro das matrizes de pesos são definidos como zero com base na sua importância.	Remove componentes inteiros da rede, como Neurónios, Embeddings ou até Head Attentions.

Tabela 15 - Tipos de Pruning

A eliminação de conexões consideradas pouco relevantes permite que o modelo se torne mais leve e eficiente, resultando numa redução do consumo de memória. Esta otimização permite igualmente uma inferência mais célere, na medida em que há um número reduzido de operações computacionais a executar durante a geração de respostas. Em determinadas circunstâncias, a técnica de Pruning pode resultar numa otimização da generalização do modelo, mediante a eliminação de parâmetros redundantes que poderiam conduzir a um ajuste excessivo. É importante salientar que esta técnica apresenta algumas limitações significativas e é frequentemente necessário implementar uma fase de otimização posterior para recuperar parte do desempenho perdido após a compressão [47].

O Pruning é um processo de simplificação do modelo, no qual se tenta manter apenas as partes mais relevantes da Rede Neural, eliminando redundâncias adquiridas durante o treino.

7.3 Knowledge Distillation

Knowledge Distillation constitui uma técnica de compressão de modelos, na qual um modelo de grandes dimensões (**teacher**) transfere o seu conhecimento para um modelo mais pequeno (**student**). A ideia central é fundamentada na premissa de que modelos de grandes dimensões não apenas captam a resposta correta, mas também identificam padrões mais ricos na distribuição das probabilidades de saída. No fundo, o modelo mais pequeno é treinado para imitar as saídas do modelo maior [48].

<i>Vantagens</i>	<i>Desvantagens</i>
Redução significativa do tamanho do modelo	Dependência de um modelo teacher de alta qualidade
Inferência mais rápida e eficiente	Processo de treino mais complexo
Adequado para deployment em dispositivos com poucos recursos	Possível perda de performance em tarefas muito complexas
Menor consumo de memória e energia	Requer alinhamento cuidadoso entre teacher e student
Preservação de boa parte do desempenho do modelo original	

Tabela 16 - Vantagens e Desvantagens de Knowledge Distillation

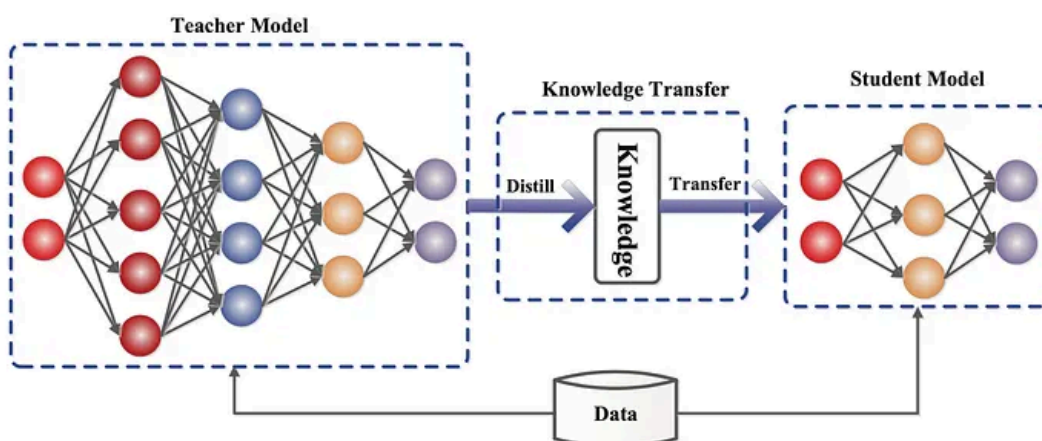


Figura 30 - Knowledge Distillation

7.4 Trade-Off entre Velocidade e Qualidade

Este equilíbrio é uma consequência direta das técnicas aplicadas para tornar os modelos mais eficientes, tais como a Quantization, o Pruning e a Knowledge Distillation. Em termos gerais, a velocidade de inferência de um modelo é diretamente proporcional ao seu tamanho e complexidade, uma vez que o número de cálculos e a quantidade de memória requeridos para gerar uma resposta são menores. Estas otimizações podem introduzir perdas na capacidade do modelo em capturar nuances linguísticas, raciocínio complexo ou detalhes contextuais, resultando numa diminuição da qualidade das respostas.

Na outra face da moeda, modelos mais abrangentes e com um maior número de parâmetros tendem a exibir um desempenho superior em tarefas complexas. No entanto, estes modelos requerem mais recursos computacionais e apresentam uma maior latência durante o processo de inferência. Deste modo, é preciso ter sempre em conta, um compromisso entre eficiência e desempenho, sendo que a escolha da abordagem depende do contexto de aplicação.

Em cenários de produção, esta relação é particularmente relevante: os sistemas que exigem respostas em tempo real privilegiam a velocidade, enquanto as aplicações críticas, onde a precisão é mais importante, tendem a favorecer modelos mais complexos e precisos.

8. [Small Language Models](#)

Incluí este tema após a apresentação de técnicas como Fine-Tuning, RAG e Model Compression, uma vez que os **Small Language Models (SLMs)** são o resultado direto da aplicação das abordagens de otimização a modelos de linguagem de grande escala.

Os SLMs consistem em modelos de linguagem que apresentam um número substancialmente menor de parâmetros em comparação com os Large Language Models (LLMs). Estes são obtidos por meio de técnicas como a *Quantization*, o *Pruning* e a *Knowledge Distillation*. Estas técnicas permitem reduzir o tamanho do modelo e os seus requisitos computacionais, mantendo um nível de desempenho aceitável para diversas tarefas de processamento de linguagem natural. Os Small Language Models são nada mais, nada menos do que uma alternativa mais leve e eficiente, especialmente adequada para ambientes com restrições de recursos computacionais [49].

Small Language Models, como a família **Phi da Microsoft** ou os modelos compactos da **Mistral AI**, demonstram que é possível alcançar elevado desempenho em tarefas de linguagem natural através de arquiteturas otimizadas e técnicas de compressão.

SLMs evidenciam-se como uma solução prática para a democratização da utilização de modelos de linguagem, permitindo a sua implementação em cenários onde os LLMs tradicionais seriam inviáveis do ponto de vista computacional ou económico.

LLMs vs. SLMs: UNDERSTANDING THE TRADE-OFFS			
	 LARGE LANGUAGE MODELS (LLMs) (The Generalist Giants)		 SMALL LANGUAGE MODELS (SLMs) (The Specialized Sprinters)
	SCALE & PARAMETERS >100 Billion Parameters.		SCALE & PARAMETERS <10 Billion Parameters.
	PRIMARY STRENGTHS Complex Reasoning, Broad World Knowledge, Novel Tasks.		PRIMARY STRENGTHS Specialized Domains, High-Volume Routine Tasks.
	DEPLOYMENT & COST Cloud API, High Variable Cost (per token).		DEPLOYMENT & COST Local/On-Premise, Fixed Low Cost (hardware).
	LATENCY (SPEED) Slower (~1s+). Network dependent.		LATENCY (SPEED) Ultra-Fast (50-200ms). Instant local response.
	DATA PRIVACY Data sent to external Cloud. Lower Privacy.		DATA PRIVACY Data stays On-Premise/Device. Complete Privacy.
BEST FOR... Complex, open-ended research & creative problem-solving.		BEST FOR... Predictable, high-volume, privacy-sensitive applications.	

Figura 31 - LLMs vs SLMs

9. Sistemas e Arquiteturas

9.1 Arquitetura Geral de um Produto LLM

O núcleo responsável pela geração de texto é o LLM. No entanto, as aplicações modernas integram diversas componentes adicionais que são responsáveis pela gestão de contexto, recuperação de informação, memória, ferramentas externas e infraestrutura de inferência [50].

Um produto LLM pode ser compreendido como uma pipeline composta por múltiplas etapas interligadas.

<i>Input do Utilizador</i>	O utilizador envia um prompt com uma instrução ou tarefa.
<i>Prompt Processing</i>	O sistema limpa o input, adiciona systems prompts, históricos de conversa e aplica regras.
<i>RAG (Opcional)</i>	Se necessário, procura em documentos, pesquisa numa vector database, obtém contexto externo.
<i>Construção de Contexto</i>	O sistema faz uma salada. Junta o input do utilizador, histórico, documentos externos e o system prompt.
<i>Inferência</i>	Após a construção de contexto, o modelo gera os tokens.

<i>Tool Calling</i>	Muito importante no mercado atual! Um modelo pode decidir pesquisar na web, usar uma calculadora, executar código ou chamar uma API.
<i>Memória</i>	Um sistema pode guardar preferência ou histórico de conversas.
<i>Geração de Resposta</i>	Resposta final do modelo!

Tabela 17 - Arquitetura Geral de um Produto LLM, mostra que os LLM's são adaptados para servirem como produto

9.2 AI Agents

Um **AI Agent** é como um sistema com a capacidade de utilizar um modelo de linguagem para não só gerar texto, mas também para tomar decisões, planejar ações e interagir autonomamente com ferramentas e ambientes externos. Em contraste com os chatbots tradicionais, que fornecem respostas diretamente a uma única instrução, os agentes são capazes de decompor tarefas complexas em múltiplas etapas intermédias, avaliando continuamente o estado atual do problema antes de decidir a ação subsequente a executar [51].

O funcionamento de um AI Agent baseia-se normalmente num ciclo iterativo composto por várias etapas:

1. Receção do objetivo ou tarefa por parte do utilizador
2. Análise do problema
3. Planeamento de ações intermédias
4. Utilização de ferramentas externas via API's
5. Observação dos resultados obtidos
6. Ajuste do plano e continuação da execução

A capacidade de planejar e raciocinar é uma das características mais relevantes dos agentes de inteligência artificial. Em vez de produzirem imediatamente uma resposta final, estes sistemas conseguem gerar etapas intermédias de raciocínio, avaliar diferentes possibilidades e decidir autonomamente qual a sequência de ações mais adequada para atingir um determinado objetivo. Por exemplo, perante uma instrução como "**Analisa o mercado de GPUs e cria um relatório comparativo**", um AI Agent pode pesquisar informação online, comparar preços, recolher benchmarks, gerar tabelas e resumir conclusões de forma sequencial e parcialmente autónoma. Com vista a expandir as capacidades do modelo, os AI Agents recorrem frequentemente a ferramentas externas. Estas ferramentas podem incluir

motores de pesquisa, APIs, bases de dados, calculadoras, ambientes de execução de código ou sistemas Retrieval-Augmented Generation. Estes modelos deixam de depender exclusivamente do conhecimento adquirido durante o período de Pre-Training e passam a interagir ativamente com o ambiente externo, permitindo o acesso a informação atualizada e a execução de operações adicionais.

Não obstante o elevado potencial destes sistemas, os AI Agents ainda apresentam diversas limitações. Destacam-se a propagação de erros ao longo das várias etapas de execução, inconsistências no planeamento, elevado consumo computacional e forte dependência da qualidade das ferramentas externas utilizadas [52].

9.3 Infraestrutura da Inferência

A implementação de Large Language Models em contextos práticos requer uma infraestrutura computacional substancialmente otimizada, comumente denominada como ***Inference Infrastructure***. Esta infraestrutura é responsável por garantir que o modelo consiga gerar respostas de forma eficiente, escalável e com baixa latência, mesmo quando existem milhares ou milhões de utilizadores a interagir simultaneamente com o sistema. Em oposição à fase de treino, a inferência opera de forma contínua durante a utilização do modelo. A otimização do processo de inferência emerge como um dos principais desafios na implementação prática de LLMs.

Dois dos conceitos mais importantes para avaliar desempenho são, a ***latência*** e o ***throughput***. São ambos conceitos cruciais, uma vez que determinam diretamente a qualidade da experiência do utilizador e a eficiência do sistema em produção.

A ***latência*** corresponde ao tempo necessário para o modelo gerar uma resposta após o receber um pedido por parte do utilizador. Esta métrica é especialmente relevante, uma vez que a geração de texto decorre de forma sequencial, token a token, o que pode introduzir atrasos perceptíveis, especialmente em modelos de grande dimensão [53].

O ***throughput*** representa o número de pedidos processados pelo sistema em uma unidade de tempo. Está diretamente relacionada com a capacidade do sistema em lidar com múltiplos utilizadores em simultâneo, sendo um fator crítico em aplicações em larga escala [54].

9.3.1 GPUs e Custos Computacionais

O crescimento exponencial dos Large Language Models resultou num aumento substancial da procura de hardware especializado, nomeadamente **GPUs** (*Graphics Processing Units*). As GPUs adquiriram uma importância fundamental no contexto do treino e da inferência de modelos modernos, devido à sua elevada capacidade de processamento paralelo em operações sobre matrizes e tensors. A utilização de LLM 's no âmbito do Pre-Training frequentemente exige infraestruturas compostas por dezenas, centenas ou até milhares de GPUs de alto desempenho, o que introduz custos extremamente elevados. As GPUs

modernas, apresentam preços elevados e a sua disponibilidade é limitada no mercado, devido à elevada procura a nível global.

Além do custo direto do hardware, é necessário considerar despesas adicionais relacionadas com o **consumo energético, refrigeração, manutenção e infraestrutura de centros de dados**.

O mercado das GPUs tornou-se um dos fatores centrais no desenvolvimento da inteligência artificial moderna, influenciando diretamente os custos, a acessibilidade e a velocidade de evolução dos sistemas baseados em modelos de linguagem.

GPU Model	Best US Price	Lowest-Ever U.S. Price	MSRP Launch Price
GeForce RTX 5090	\$3,799	\$1,999	\$1,999
GeForce RTX 5080	\$1,289	\$929	\$999
GeForce RTX 5070 Ti	\$899	\$729	\$749
GeForce RTX 5070	\$629	\$479	\$549
GeForce RTX 5060 Ti 16GB	\$539	\$379	\$429
GeForce RTX 5060 Ti 8GB	\$379	\$319	\$379
GeForce RTX 5060	\$349	\$279	\$299
GeForce RTX 5050	\$289	\$229	\$249

Figura 32 - Preço da GPU Nvidia RTX 50-Series

9.3.2 Técnicas para Mitigar Custos Computacionais na Inferência

<i>Batching</i>	<i>KV Cache</i>
Em vez de processar apenas um pedido de cada vez, o sistema agrupa múltiplas requests de diferentes utilizadores num único batch.	Armazena representações intermédias previamente calculadas durante a Attention.
Permite maximizar a utilização do hardware disponível.	Permite reutilizar informação já processada em vez de recalcular tudo novamente a cada novo token gerado.

Tabela 18 - Técnicas para Mitigar Custos Computacionais

9.3.3 Importância da Inferência

A inferência corresponde ao momento em que o modelo é efetivamente utilizado por utilizadores finais. Enquanto o treino é um processo realizado uma única vez (ou de forma periódica), a inferência ocorre de forma contínua em sistemas em produção, sendo responsável pela geração de respostas em tempo real.

A qualidade da experiência do utilizador depende diretamente da eficiência do processo de inferência. Mesmo os modelos com elevado desempenho durante o treino podem tornar-se impraticáveis se a sua inferência for demasiado lenta ou dispendiosa.

A inferência constitui o ponto em que diversas otimizações à arquitetura de um modelo se tornam mais pertinentes, tais como a utilização de **KV Cache**, **Batching** e **Quantization**. Estas otimizações não alteram o modelo em si, mas permitem torná-lo mais eficiente e escalável em ambientes reais.

10. Deployment e Produção - Servidores, Web e Cloud Computing

A disponibilização de LLM's ou SLM's em ambientes de produção exige uma infraestrutura capaz de suportar pedidos contínuos, elevada carga computacional e necessidade de baixa latência. Para tal, os modelos são, habitualmente, hospedados através de servidores dedicados, aplicações web ou plataformas de computação em nuvem.

10.1 Servidores

A hospedagem em servidores próprios consiste na execução do modelo em infraestrutura controlada pela organização. Estes servidores podem ser físicos (**on-premise**) ou máquinas dedicadas alugadas.

Neste sistema:

- O controlo total do sistema é da entidade que o implementa
- Há maior flexibilidade na configuração do hardware e otimização
- Existe maior responsabilidade em manutenção, escalabilidade e segurança

Esta abordagem é excelente para empresas que necessitam de controlo total sobre dados sensíveis ou que possuem infraestruturas de alto desempenho.

10.2 Web Deployment (API's)

Na hospedagem via web, o modelo é disponibilizado através de uma API, permitindo que aplicações externas comuniquem com o modelo através de pedidos HTTP.

Características:

- O modelo funciona como um serviço (*Model as a Service*)
- O utilizador final não interage diretamente com o modelo
- A integração é feita em aplicações Web, Mobile ou Backend

Este é o modelo mais comum atualmente, pois simplifica a integração e abstrai toda a complexidade da infraestrutura.

10.3 Cloud Computing

O **Cloud Computing** é o meio prevalecente para a hospedagem de LLMs ou SLMs, com plataformas como AWS, Google Cloud ou Azure a desempenharem um papel preponderante neste domínio.

As vantagens principais incluem a escalabilidade automática em conformidade com o número de utilizadores, o acesso a GPUs de alto desempenho sem necessidade de investimento inicial, alta disponibilidade e redundância e facilidade de implementação global.

11. Ferramentas e Tecnologias Importantes

<i>Linguagens de Programação e Frameworks</i>	Python	Pytorch	Tensor Flow
<i>Ambientes de Desenvolvimento</i>	Google Colab	Kaggle Notebooks	Hugging Face Spaces & Notebooks
<i>Ecossistema de Modelos</i>	Hugging Face	LLaMA	Mistral
<i>Fine-Tuning</i>	Hugging Face Ecosystem	Unsloth	Axolotl
<i>Prompt Engineering</i>	LangChain	LlamaIndex	
<i>RAG</i>	FAISS / Pinecone	LangChain	LlamaIndex
<i>Model Compression</i>	bitsandbytes / GPTQ	FlashAttention	vLLM / TensorRT-LLM
<i>Inferência</i>	vLLM	TensorRT-LLM	LitGPT
<i>Hospedagem</i>	Ollama / LM Studio / llama.cpp	Docker / FastAPI	Cloud
<i>AI Agents</i>	n8n	Agentx	Zapier

Tabela 19 - Ferramentas e Tecnologias Importantes

12. Tecnologias Emergentes

As tecnologias emergentes no domínio dos modelos de linguagem estão a evoluir a um ritmo acelerado, ultrapassando os limites dos tradicionais sistemas de processamento de linguagem natural (LLM).

<i>Small Language Models</i>	Modelos cada vez menores mas cada vez mais otimizados para inferência.
<i>Agentic AI</i>	LLMs deixam de ser “chatbots” e tornam-se agentes autónomos capazes de planear e executar tarefas.
<i>Multimodal Models</i>	Modelos que processam múltiplos tipos de dados.
<i>Mixture of Experts (MoE)</i>	Arquitetura onde apenas partes do modelo são ativadas por input.
<i>Long Context Models</i>	Modelos com capacidade mais longa de contexto.
<i>RAG Evoluído</i>	Evolução do RAG tradicional.
<i>Model Compression + Edge IA</i>	Levar LLMs para telemóveis, browsers e dispositivos locais.
<i>Modelos Self-Improving</i>	Modelos que ajustam comportamento conforme o feedback.
<i>Geração de Dados Sintéticos</i>	LLMs geram dados para treinar outros LLMs.
<i>Model Routing & Multi-Model Systems</i>	Sistemas escolhem dinamicamente qual modelo usar. SLM para tarefas simples. LLM para tarefas complexas.
<i>AI Systems + API's</i>	LLMs deixam de ser modelos isolados e tornam-se controladores de sistemas.

Tabela 20 - Tecnologias Emergentes